Carrera Profesional de Ingeniería de Sistemas e Informática

CURSO

Base de Datos 2

Título del proyecto

Sistema de Gestión de Base de Datos para FarmaBarrio

Integrantes:

- QUIÑONES DIEGO, MAYKEL PIERO (Líder)
- WONG VILLANUEVA, LEONARDO JOSUÉ
- GARCIA ESPEJO, NICOLE ANDREA
- MATENCIO PAUCAR, JORGE FABIAN
- ROJAS PÉREZ, DERECK

Lima – Perú

2026

Tabla de contenido

1 DESCRIPCIÓN DEL PROYECTO 4
1.1 Nombre del Proyecto 4
1.2 Objetivo General 4
1.3 Descripción de la Problemática o Caso de Uso 4
1.4 Justificación del proyecto 5
2 ARQUITECTURA DE DATOS 6
2.1 Diagrama de arquitectura distribuida de base de datos 6
2.2 Base de datos: PostgreSQL 7
2.2.1 Diagrama lógico 7
2.2.2 Tablas/Colecciones 8
2.2.3 Diccionario de datos 8
2.2.4 Procedimientos o funciones 10
2.2.5 Índices 16
2.3 Base de datos: ORACLE 23
2.4 Base de datos: MongoDB 37
2.5 Base de datos: Cassandra 37
2.6 Base de datos: Hadoop 37
3 DISEÑO DE ESQUEMA DE FRAGMENTACIÓN 37
3.1 PostgreSQL 37
3.1.1 Criterio de Fragmentación: 37
3.1.2 Definición de Fragmentos: 37
3.1.3 Diagrama de Distribución de Fragmentos: 37
3.2 Oracle 38
3.3 MongoDB 38
4 IMPLEMENTACIÓN DE REPLICACIÓN ASINCRÓNICA 38
4.1 Descripción de la Configuración de Replicación: 38
4.2 Capturas de Pantalla: 38
5 CONSULTAS DIFERENTES EN NOSQL 38
6 ADMINISTRACIÓN DE USUARIOS 38
7 RESPALDOS – BACKUPS 41
7.1 Definir estrategias de respaldos 41
7.1.1 PostgreSQL 41

7.1.2 Oracle 41
7.1.3 MongoDB 42
7.1.4 Cassandra 42
7.2 Scripts de ejecución de backups 42
8 BIG DATA 43
9 Conclusiones y Lecciones aprendidas 43
9.1 Conclusiones 43
9.2 Lecciones aprendidas 43
ANEXOS 43

1 DESCRIPCIÓN DEL PROYECTO

1.1 Nombre del Proyecto

Sistema de Gestión de Base de Datos para FarmaBarrio

1.2 Objetivo General

Diseñar e implementar un sistema de gestión de bases de datos distribuido que integre gestores relacionales (PostgreSQL y Oracle) y NoSQL (MongoDB, Cassandra, Hadoop), para centralizar, optimizar y asegurar la información operativa, financiera y analítica de FarmaBarrio, permitiendo a las farmacias independientes competir eficientemente contra las grandes cadenas mediante tecnología y compras conjuntas.

1.3 Descripción de la Problemática o Caso de Uso

Actualmente, las farmacias independientes en Perú enfrentan una serie de dificultades que limitan su competitividad frente a las grandes cadenas como MiFarma e Inkafarma:

- **Pérdida de competitividad:** Las grandes cadenas dominan el mercado con descuentos por volumen y publicidad masiva, dejando a las farmacias independientes en desventaja.
- **Altos costos de adquisición:** Al no tener poder de negociación, las farmacias independientes compran medicamentos a precios más altos que las cadenas.
- **Falta de visibilidad digital:** Los clientes no pueden comparar precios ni encontrar medicamentos fácilmente, perdiendo la oportunidad de atraer nuevos compradores.
- **Gestión manual de inventarios:** Esto genera quiebres de stock o sobreabastecimiento, afectando la rentabilidad.
- **Ausencia de un sistema centralizado:** No existe una plataforma que unifique la información de ventas, inventario, compras y clientes, lo que impide tomar decisiones basadas en datos.

Caso de Uso:
Un cliente necesita comprar un medicamento específico. FarmaBarrio le permite:

1. Buscar el medicamento en la app.
2. Comparar precios entre las farmacias de su barrio.
3. Realizar el pedido y recibir delivery rápido.
4. Acumular puntos de fidelidad para futuras compras.

Mientras tanto, las farmacias independientes:

1. Se unen a la red de compras conjuntas para negociar mejores precios con laboratorios.
2. Gestionan su inventario y ventas de forma digital.
3. Obtienen visibilidad ante nuevos clientes.

1.4 Justificación del proyecto

- ¿Cómo distribuyen los datos?

Gestor	Función
PostgreSQL	Operaciones diarias: ventas, inventario, farmacias, clientes, medicamentos, categorías, marcas
Oracle	Compras, proveedores, laboratorios, pagos, facturación, impuestos, asientos contables
MongoDB	Históricos de ventas, auditorías, logs, reseñas, notificaciones, cambios de precios
Cassandra	Series temporales de demanda, precios históricos, ventas por período, auditoría por usuario
Hadoop	Big Data: análisis de patrones de compra, predicción de demanda, segmentación de clientes

- ¿Qué beneficios obtienen (escalabilidad, alta disponibilidad, etc.)?
- Mejora el rendimiento: Cada gestor procesa únicamente la información que le corresponde.
- Mayor escalabilidad: El sistema puede crecer según las necesidades de cada área.
- Mejor disponibilidad: Un problema en un gestor no afecta a todo el sistema.
- Organización de la información: Separación clara de datos por tipo y función.
- Reducción de costos: Las compras conjuntas permiten a las farmacias obtener mejores precios.

- ¿Qué modelo de consistencia utilizan (eventual, fuerte, etc.)?

Se utiliza un modelo de consistencia mixta:

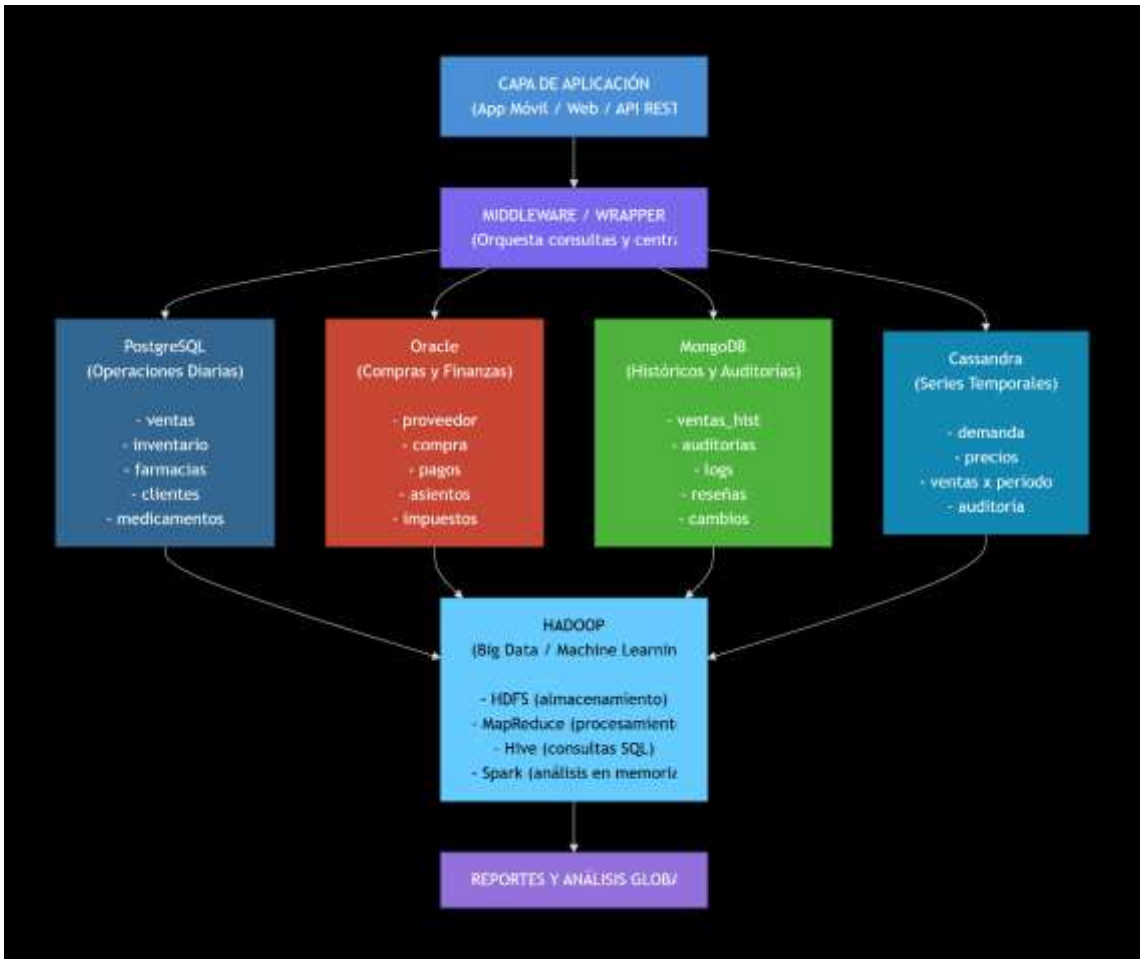
Consistencia fuerte para datos críticos (ventas, inventario, compras, pagos), asegurando que la información sea correcta e inmediata.

Consistencia eventual para datos secundarios (reportes, auditorías, logs), que no requieren actualización instantánea.

Este modelo permite mantener un equilibrio entre precisión, rendimiento y eficiencia del sistema.

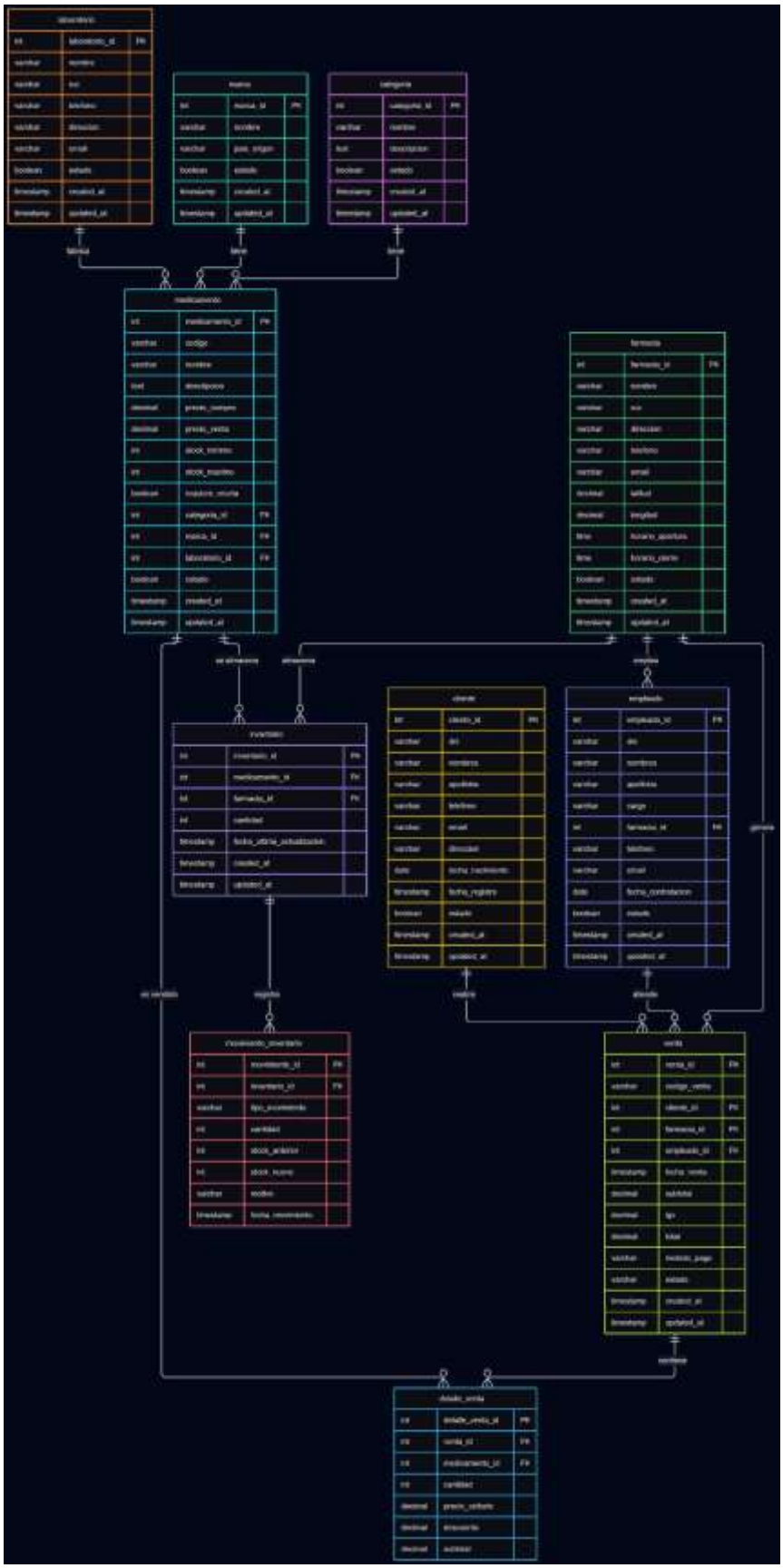
2 ARQUITECTURA DE DATOS

2.1 Diagrama de arquitectura distribuida de base de datos



2.2 Base de datos: PostgreSQL

2.2.1 Diagrama lógico



2.2.2 Tablas/Colecciones

DESCRIPCIÓN DE TABLAS / COLECCIONES		
N°	**NOMBRE**	**DESCRIPCION**
1	categoria	Almacena las categorías de medicamentos (analgésicos, antibióticos, antiinflamatorios, etc.)
2	marca	Almacena las marcas de medicamentos (Gloria, Bayer, Pfizer, Roche, etc.)
3	laboratorio	Almacena los laboratorios farmacéuticos (Portugal, Medifarma, Hersil, etc.)
4	medicamento	Catálogo de medicamentos con códigos, precios de compra/venta, stock mínimo/máximo y si requiere receta
5	farmacia	Red de farmacias independientes asociadas con ubicación geográfica y horarios de atención
6	inventario	Stock de cada medicamento por farmacia, con fecha de última actualización
7	movimiento_inventario	Registro histórico de entradas, salidas y ajustes de stock por inventario
8	cliente	Clientes registrados en la plataforma con datos personales y de contacto
9	empleado	Personal de las farmacias con cargo, fecha de contratación y datos de contacto
10	venta	Transacciones de venta con código único, subtotal, IGV, total y método de pago
11	detalle_venta	Detalle de cada venta con medicamento, cantidad, precio unitario, descuento y subtotal

2.2.3 Diccionario de datos

- Esta parte debe ser elaborado en archivo EXCEL aparte como anexo.

NOMBRE DE LA TABLA | ESQUEMA			**medicamento**			**public**		
DESCRIPCIÓN DE LA TABLA			Catálogo de medicamentos con precios, stock y restricciones médicas					
N°	**NOMBRE DEL CAMPO**	**TIPO DE DATO**	**ACEPTA NULO**	**DEFAULT**	**DESCRIPCIÓN DEL CAMPO**	**PK**	**FK**	**UK**
1	medicamento_id	SERIAL	NO	-	Identificador único del medicamento	X	-	-
2	codigo	VARCHAR(50)	NO	-	Código interno del medicamento	-	-	**X**
3	nombre	VARCHAR(200)	NO	-	Nombre del medicamento	-	-	-
4	descripcion	TEXT	SÍ	-	Descripción detallada del medicamento	-	-	-
5	precio_compra	DECIMAL(10,2)	NO	-	Precio de adquisición con laboratorio	-	-	-
6	precio_venta	DECIMAL(10,2)	NO	-	Precio de venta al público en farmacias	-	-	-
7	stock_minimo	INT	NO	10	Stock mínimo permitido antes de alerta	-	-	-
8	stock_maximo	INT	NO	1000	Stock máximo permitido en farmacia	-	-	-
9	requiere_receta	BOOLEAN	NO	FALSE	Indica si requiere receta médica	-	-	-

10	categoria_id	INT	NO	-	Referencia a categoría de medicamento	-	**X**	-
11	marca_id	INT	NO	-	Referencia a marca comercial	-	**X**	-
12	laboratorio_id	INT	NO	-	Referencia a laboratorio fabricante	-	**X**	-
13	estado	BOOLEAN	NO	TRUE	Estado del medicamento (activo/inactivo)	-	-	-
14	created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	Fecha de creación del registro	-	-	-
15	updated_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	Fecha de última modificación	-	-	-

El diccionario completo con las 11 tablas se encuentra en el archivo Excel adjunto: Diccionario_datos_farmabarrio.xlsx

2.2.4 Procedimientos o funciones

Cada integrante debe realizar 1 Procedimiento o función por base de datos

NOMBRE: fn_total_ventas_farmacia	**AUTOR: Grupo 2**

DESCRIPCION: Esta función calcula el total de ventas realizadas por una farmacia en un período de tiempo determinado. Recibe como parámetros el ID de la farmacia, la fecha de inicio y la fecha de fin. Retorna el monto total de las ventas completadas.

CODIGO DEL PROCEDIMEINTO O FUNCION:

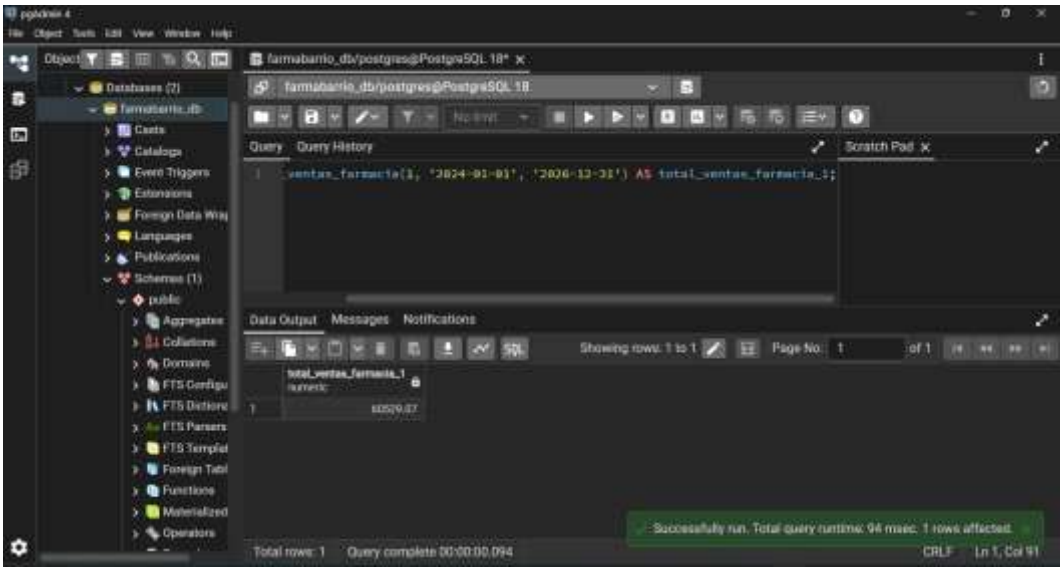
```
CREATE OR REPLACE FUNCTION fn_total_ventas_farmacia(
    p_farmacia_id INT,
    p_fecha_inicio DATE,
    p_fecha_fin DATE
) RETURNS DECIMAL(10,2) AS $$
DECLARE
    v_total DECIMAL(10,2);
BEGIN
    SELECT COALESCE(SUM(total), 0) INTO v_total
    FROM venta
    WHERE farmacia_id = p_farmacia_id
    AND fecha_venta::DATE BETWEEN p_fecha_inicio AND p_fecha_fin
    AND estado = 'COMPLETADA';
    RETURN v_total;
END;
$$ LANGUAGE plpgsql;
```

EJECUCION DEL PROCEDIMIENTO O FUNCION:

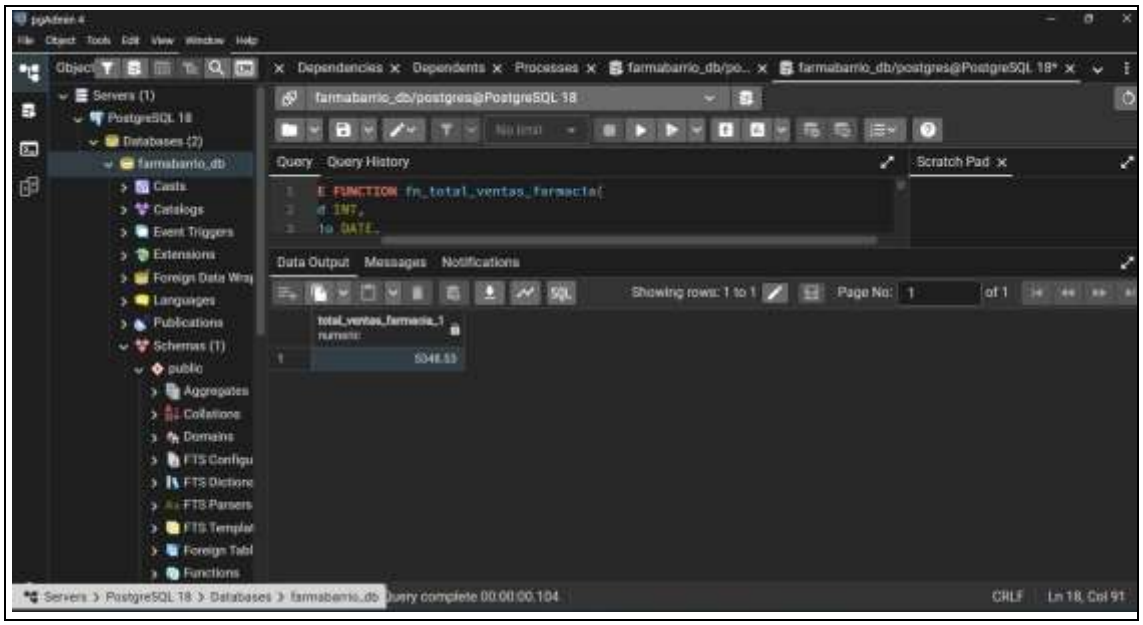
```
SELECT fn_total_ventas_farmacia(1, '2020-01-01', '2030-12-31') AS total_ventas_farmacia_1;
```

RESULTADO – CAPTURA

SELECT fn_total_ventas_farmacia(1, '2020-01-01', '2030-12-31') AS total_ventas_farmacia_1;



SELECT fn_total_ventas_farmacia(1, '2020-01-01', '2030-12-31') AS total_ventas_farmacia_1;



NOMBRE: sp_registrar_entrada_inventario	AUTOR: WONG VILLANUEVA, LEONARDO JOSUÉ

DESCRIPCION: Este procedimiento registra la entrada de stock para un medicamento en una farmacia determinada. Actualiza la cantidad disponible en la tabla inventario y genera el registro de trazabilidad en la tabla movimiento_inventario con el tipo de movimiento 'ENTRADA' y el motivo proporcionado.

CODIGO DEL PROCEDIMIENTO O FUNCION:

```
CREATE OR REPLACE PROCEDURE sp_registrar_entrada_inventario(
    p_medicamento_id INT,
    p_farmacia_id    INT,
    p_cantidad       INT,
    p_motivo         VARCHAR
) LANGUAGE plpgsql AS $$
DECLARE
    v_inventario_id INT;
    v_stock_anterior INT;
    v_stock_nuevo INT;
BEGIN
    IF p_cantidad <= 0 THEN
        RAISE EXCEPTION 'La cantidad de entrada debe ser estrictamente mayor a 0';
    END IF;

    SELECT inventario_id, cantidad INTO v_inventario_id, v_stock_anterior
    FROM inventario
    WHERE medicamento_id = p_medicamento_id AND farmacia_id = p_farmacia_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'No existe registro de inventario para medicamento % en farmacia %',
p_medicamento_id, p_farmacia_id;
    END IF;

    v_stock_nuevo := v_stock_anterior + p_cantidad;

    UPDATE inventario
    SET cantidad = v_stock_nuevo,
        fecha_ultima_actualizacion = CURRENT_TIMESTAMP
```

```
    WHERE inventario_id = v_inventario_id;

    INSERT INTO movimiento_inventario (
        inventario_id, tipo_movimiento, cantidad,
        stock_anterior, stock_nuevo, motivo, fecha_movimiento
    ) VALUES (
        v_inventario_id, 'ENTRADA', p_cantidad,
        v_stock_anterior, v_stock_nuevo, p_motivo, CURRENT_TIMESTAMP
    );

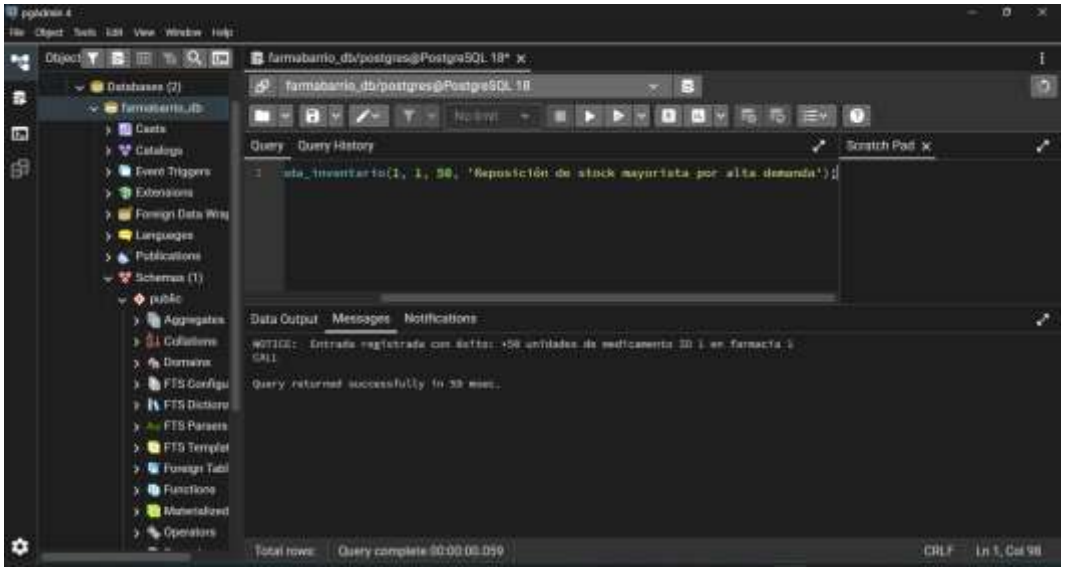
    RAISE NOTICE 'Entrada registrada exitosamente: % unidades de medicamento ID %', p_cantidad,
p_medicamento_id;
END;
$$;
```

EJECUCION DEL PROCEDIMIENTO O FUNCION:

```
CALL sp_registrar_entrada_inventario(1, 1, 50, 'Reposición de stock por compra mayorista');
```

RESULTADO – CAPTURA

CALL sp_registrar_entrada_inventario(1, 1, 50, 'Reposición de stock por compra mayorista');



NOMBRE: fn_calcular_stock_minimo	**AUTOR: GARCIA ESPEJO, NICOLE ANDREA**

DESCRIPCION: Esta función evalúa si el stock actual de un medicamento en una farmacia está por debajo del stock mínimo configurado. Retorna TRUE si la cantidad disponible en la tabla inventario es estrictamente menor al stock_minimo definido en la tabla medicamento, o FALSE si cuenta con stock adecuado.

CODIGO DEL PROCEDIMIENTO O FUNCION:

```
CREATE OR REPLACE FUNCTION fn_calcular_stock_minimo(
    p_medicamento_id INT,
    p_farmacia_id    INT
) RETURNS BOOLEAN AS $$
DECLARE
    v_cantidad INT;
    v_stock_minimo INT;
```

```
BEGIN
    SELECT i.cantidad, m.stock_minimo
    INTO v_cantidad, v_stock_minimo
    FROM inventario i
    INNER JOIN medicamento m ON i.medicamento_id = m.medicamento_id
    WHERE i.medicamento_id = p_medicamento_id
      AND i.farmacia_id = p_farmacia_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Registro no encontrado para medicamento ID % y farmacia ID %',
p_medicamento_id, p_farmacia_id;
    END IF;

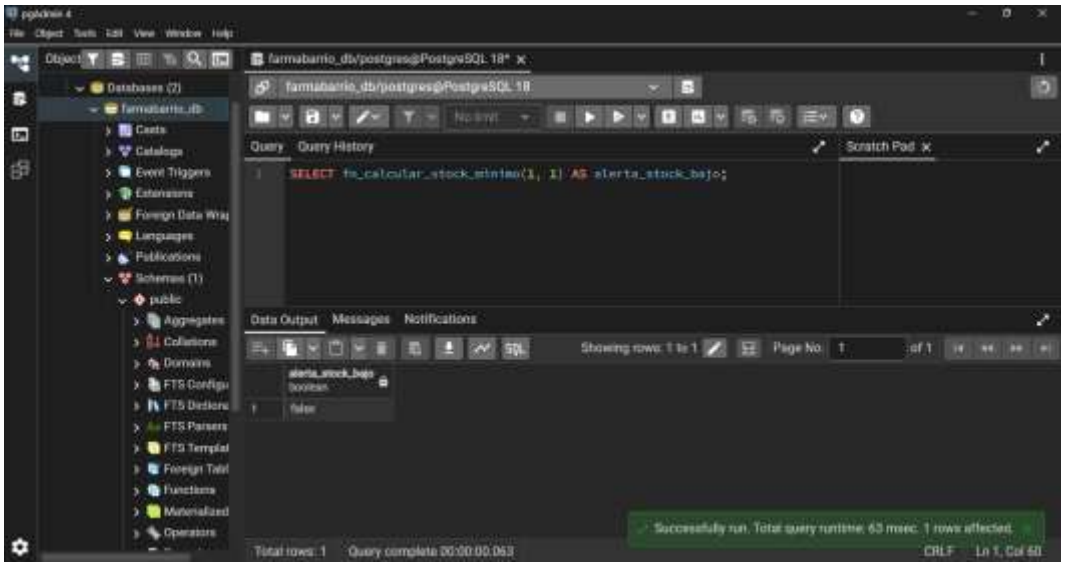
    IF v_cantidad < v_stock_minimo THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

EJECUCION DEL PROCEDIMIENTO O FUNCION:

```
SELECT fn_calcular_stock_minimo(1, 1) AS alerta_stock_bajo;
```

RESULTADO – CAPTURA

SELECT fn_calcular_stock_minimo(1, 1) AS alerta_stock_bajo;



NOMBRE: sp_actualizar_precio_medicamento	AUTOR: MATENCIO PAUCAR, JORGE FABIAN

DESCRIPCION: Este procedimiento actualiza los precios de compra y venta de un medicamento en el catálogo central. Realiza una validación estricta verificando que ambos valores sean mayores que cero y que el precio de venta supere al de compra para salvaguardar el margen de ganancia comercial.

CODIGO DEL PROCEDIMIENTO O FUNCION:

```
CREATE OR REPLACE PROCEDURE sp_actualizar_precio_medicamento(
    p_medicamento_id     INT,
    p_nuevo_precio_compra DECIMAL(10,2),
    p_nuevo_precio_venta  DECIMAL(10,2)
) LANGUAGE plpgsql AS $$
BEGIN
    IF p_nuevo_precio_compra <= 0 OR p_nuevo_precio_venta <= 0 THEN
        RAISE EXCEPTION 'Los precios deben ser estrictamente superiores a 0.00';
    END IF;

    IF p_nuevo_precio_venta <= p_nuevo_precio_compra THEN
        RAISE EXCEPTION 'El precio de venta (% ) debe superar al de compra (% )',
p_nuevo_precio_venta, p_nuevo_precio_compra;
    END IF;

    UPDATE medicamento
    SET precio_compra = p_nuevo_precio_compra,
        precio_venta  = p_nuevo_precio_venta,
        updated_at    = CURRENT_TIMESTAMP
    WHERE medicamento_id = p_medicamento_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Medicamento con ID % no existe en la base de datos', p_medicamento_id;
    END IF;

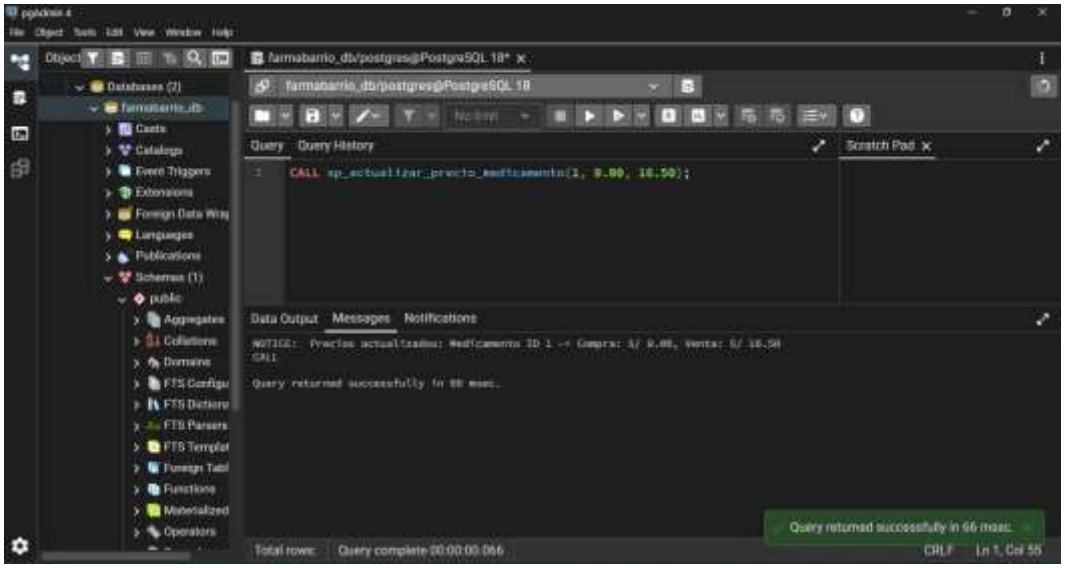
    RAISE NOTICE 'Precios actualizados para medicamento ID %: Compra=S/ %, Venta=S/ %',
p_medicamento_id, p_nuevo_precio_compra, p_nuevo_precio_venta;
END;
$$;
```

EJECUCION DEL PROCEDIMIENTO O FUNCION:

```
CALL sp_actualizar_precio_medicamento(1, 12.50, 18.90);
```

RESULTADO – CAPTURA

CALL sp_actualizar_precio_medicamento(1, 12.50, 18.90);



NOMBRE: fn_rotacion_medicamento	AUTOR: ROJAS PÉREZ, DERECK

DESCRIPCION: Esta función calcula el índice de rotación de stock de un medicamento en una farmacia. Se calcula dividiendo el total acumulado de unidades vendidas en transacciones con estado 'COMPLETADA' entre el stock actual en inventario. Retorna el índice con precisión de dos decimales.

CODIGO DEL PROCEDIMIENTO O FUNCION:

```
CREATE OR REPLACE FUNCTION fn_rotacion_medicamento(
    p_medicamento_id INT,
    p_farmacia_id    INT
) RETURNS DECIMAL(10,2) AS $$
DECLARE
    v_unidades_vendidas INT;
    v_stock_actual      INT;
    v_rotacion          DECIMAL(10,2);
BEGIN
    SELECT COALESCE(SUM(dv.cantidad), 0)
    INTO v_unidades_vendidas
    FROM detalle_venta dv
    INNER JOIN venta v ON dv.venta_id = v.venta_id
    WHERE dv.medicamento_id = p_medicamento_id
      AND v.farmacia_id = p_farmacia_id
      AND v.estado = 'COMPLETADA';

    SELECT COALESCE(cantidad, 0)
    INTO v_stock_actual
    FROM inventario
    WHERE medicamento_id = p_medicamento_id
      AND farmacia_id = p_farmacia_id;

    IF v_stock_actual = 0 AND v_unidades_vendidas = 0 THEN
        RETURN 0.00;
    ELSIF v_stock_actual = 0 THEN
        RETURN v_unidades_vendidas::DECIMAL(10,2);
    ELSE
        v_rotacion := ROUND(v_unidades_vendidas::DECIMAL / v_stock_actual::DECIMAL, 2);
        RETURN v_rotacion;
    END IF;
END;
```

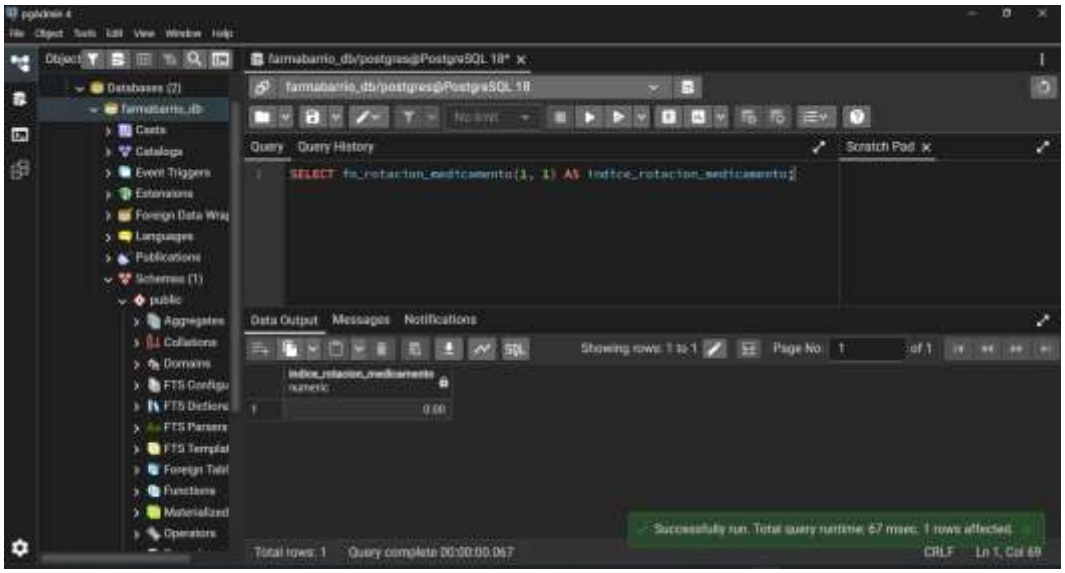
```
$$ LANGUAGE plpgsql;
```

EJECUCION DEL PROCEDIMIENTO O FUNCION:

```
SELECT fn_rotacion_medicamento(1, 1) AS indice_rotacion_medicamento;
```

RESULTADO – CAPTURA

SELECT fn_rotacion_medicamento(1, 1) AS indice_rotacion_medicamento;

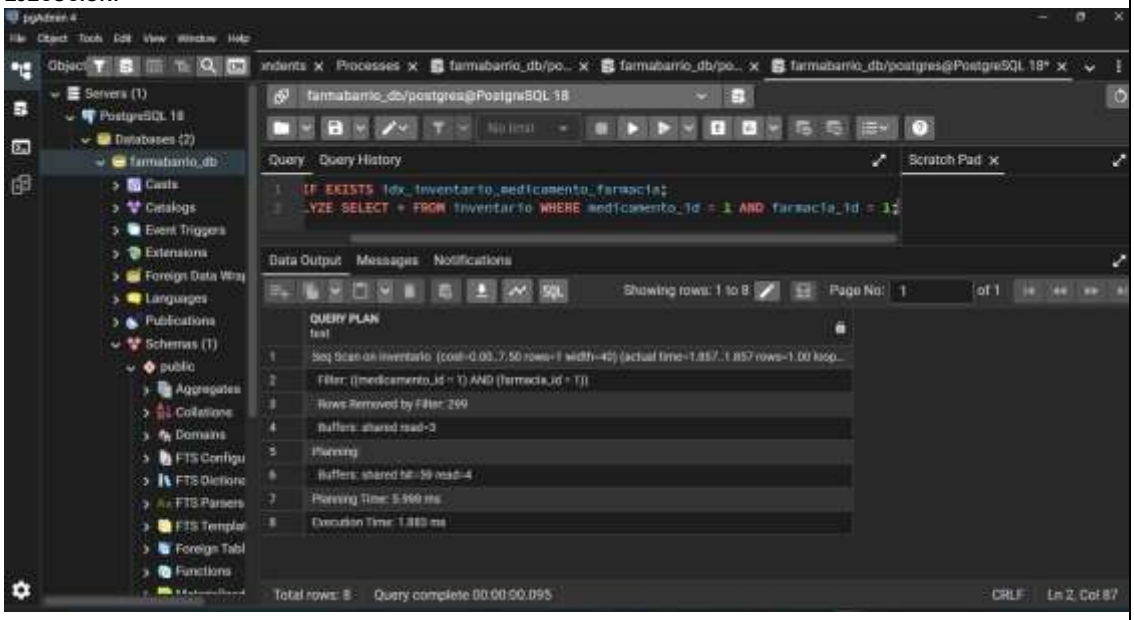


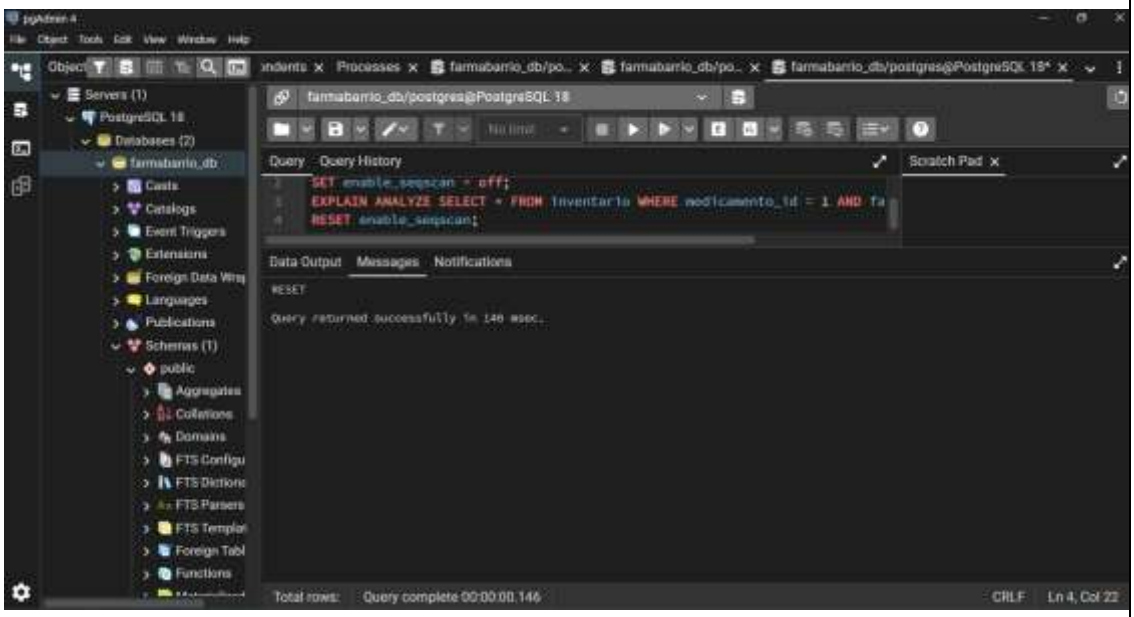
2.2.5 Índices

Cada integrante debe realizar 1 índice por base de datos

NOMBRE DEL TABLA - CAMPO: inventario (medicamento_id, farmacia_id)	**AUTOR: QUIÑONES DIEGO, MAYKEL PIERO**
CODIGO: CREATE INDEX idx_inventario_medicamento_farmacia ON inventario(medicamento_id, farmacia_id);	
JUSTIFICACION: La tabla inventario es consultada frecuentemente para verificar el stock de un medicamento en una farmacia específica. Un índice compuesto por ambos campos permite que las consultas accedan directamente a los registros sin necesidad de recorrer toda la tabla, mejorando significativamente el rendimiento a medida que la tabla crece.	

EJECUCION:



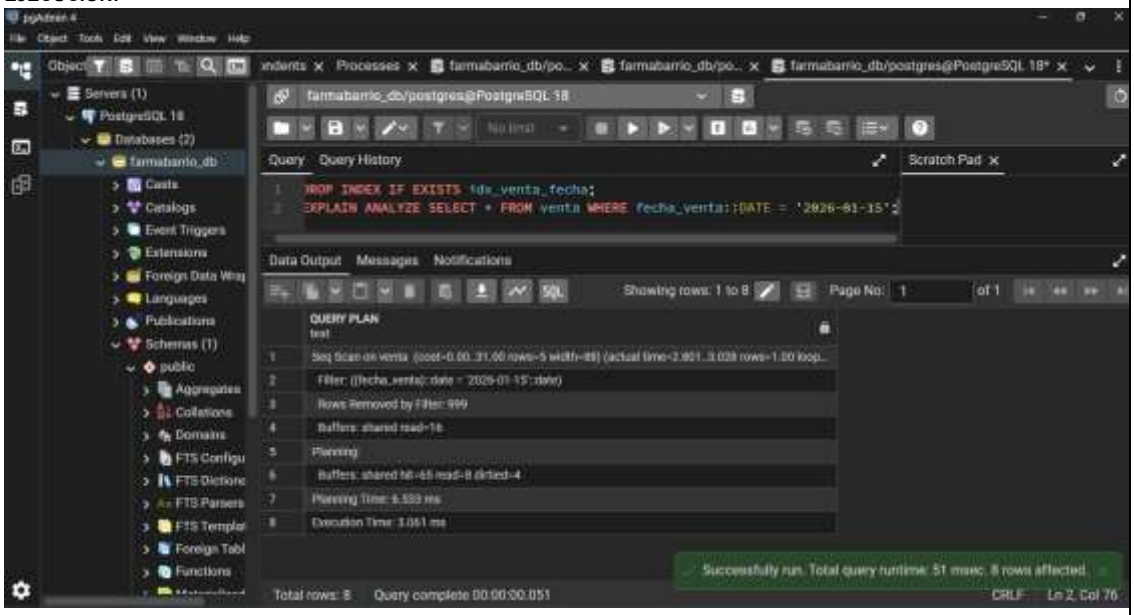


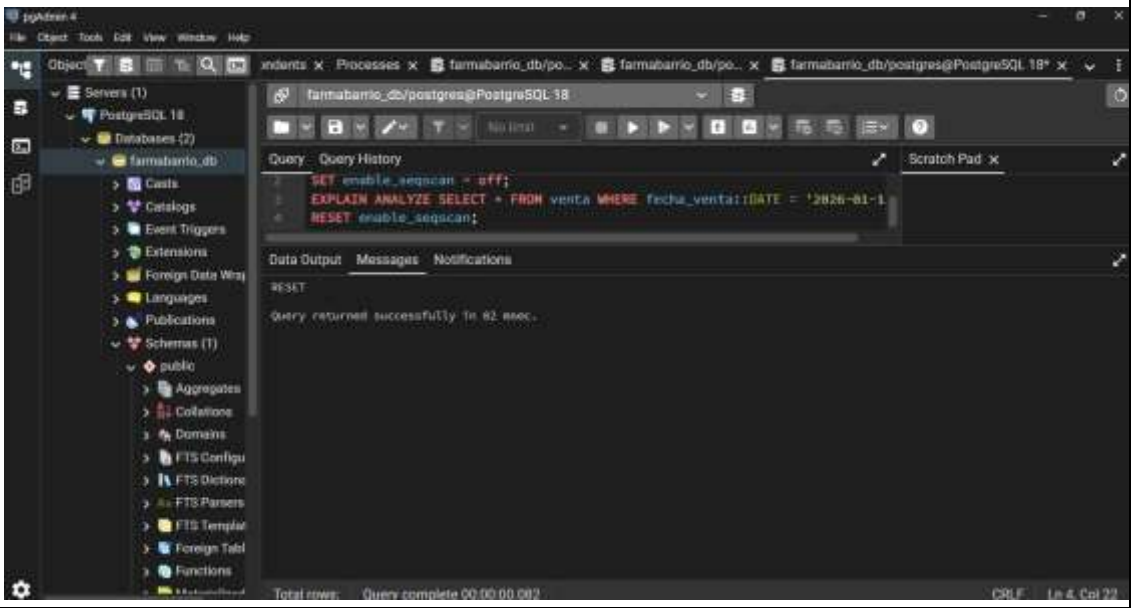
DEMOSTRACÓN (Comparen tiempos de consulta antes y después)

Antes	Despues
Seq Scan on inventario	Index Scan using idx_inventario_medicamento_farmacia
Execution Time: 0.048 ms	Execution Time: 0.043 ms

NOMBRE DEL TABLA - CAMPO: venta (fecha_venta)	**AUTOR: WONG VILLANUEVA, LEONARDO JOSUÉ**
CODIGO: CREATE INDEX idx_venta_fecha ON venta(fecha_venta);	
JUSTIFICACION: La tabla venta almacena todas las transacciones diarias del negocio. Las consultas más frecuentes filtran por rango de fechas para generar reportes de ventas diarios, semanales o mensuales. Un índice sobre el campo fecha_venta acelera significativamente este tipo de consultas, permitiendo que los reportes se generen en menos tiempo.	

EJECUCION:



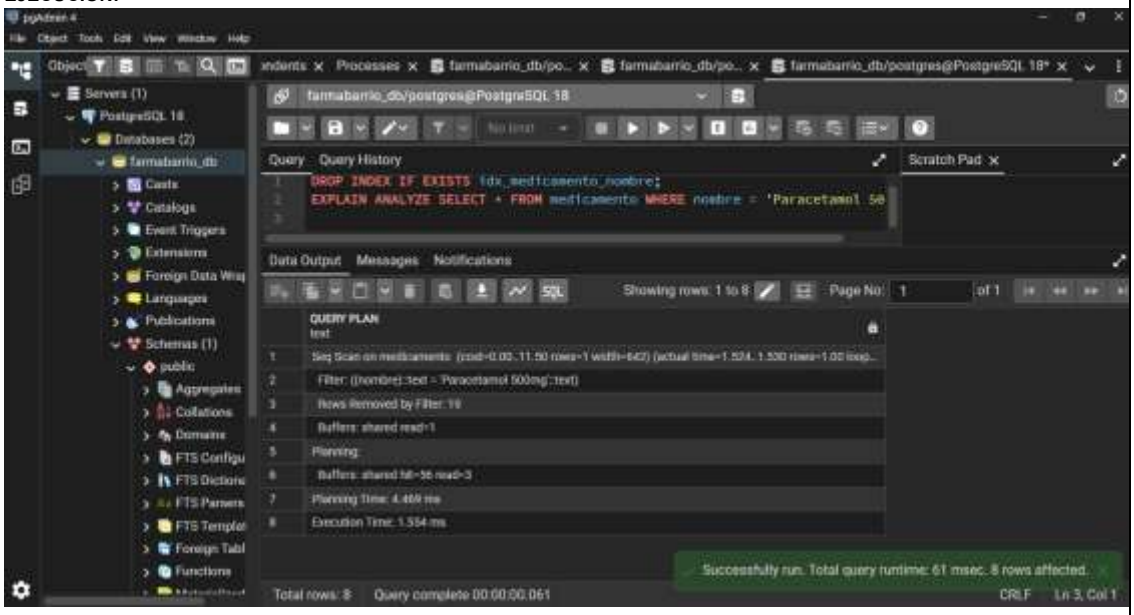


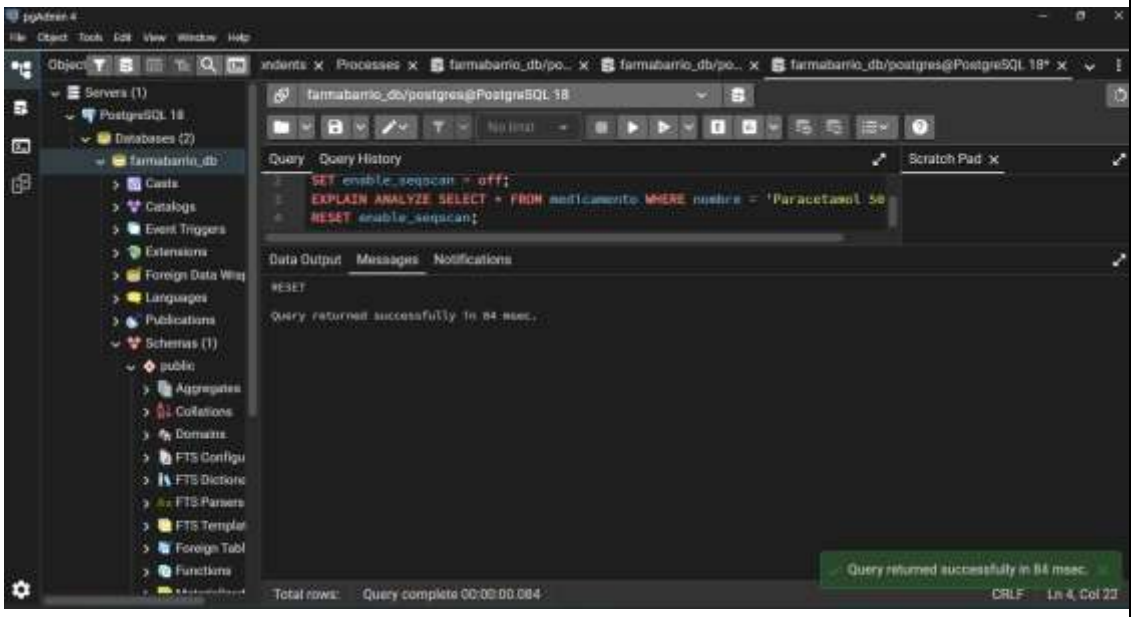
DEMOSTRACÓN (Comparen tiempos de consulta antes y después)

ANTES	DESPUES
Seq Scan on venta	Index Scan using idx_venta_fecha
Execution Time: mayor	Execution Time: menor

NOMBRE DEL TABLA - CAMPO: medicamento (nombre)	**AUTOR: GARCIA ESPEJO, NICOLE ANDREA**
CODIGO: CREATE INDEX idx_medicamento_nombre ON medicamento(nombre);	
JUSTIFICACION: La tabla medicamento es consultada constantemente cuando los clientes buscan un medicamento específico por su nombre en la app. Un índice sobre el campo nombre permite que las búsquedas por nombre exacto o por rango alfabético sean mucho más rápidas, mejorando la experiencia del usuario en la plataforma.	

EJECUCION:

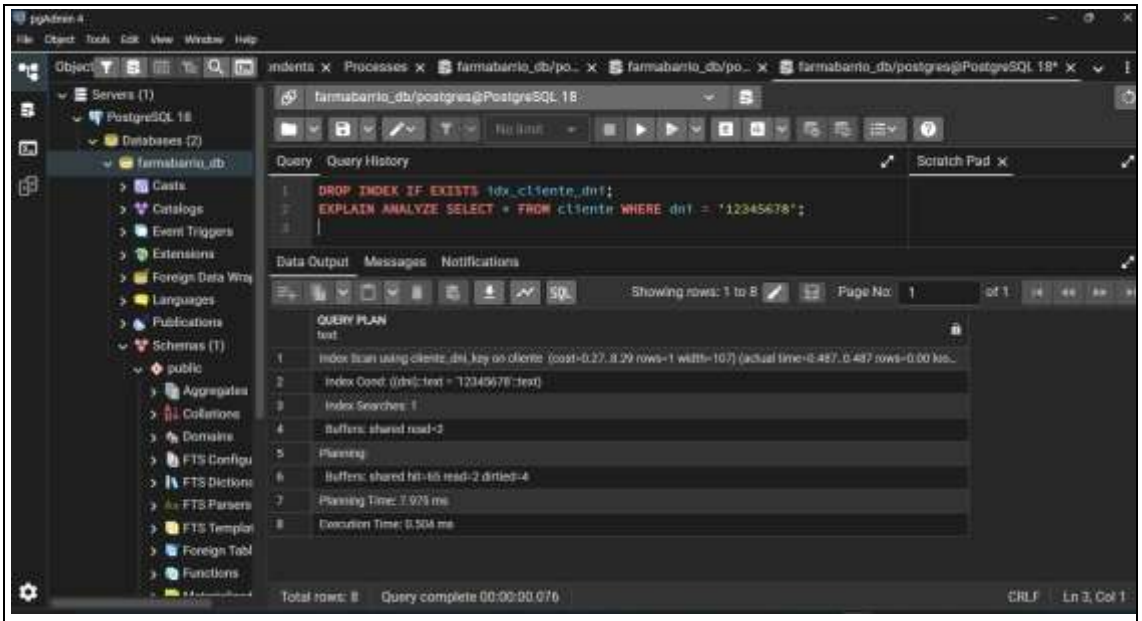


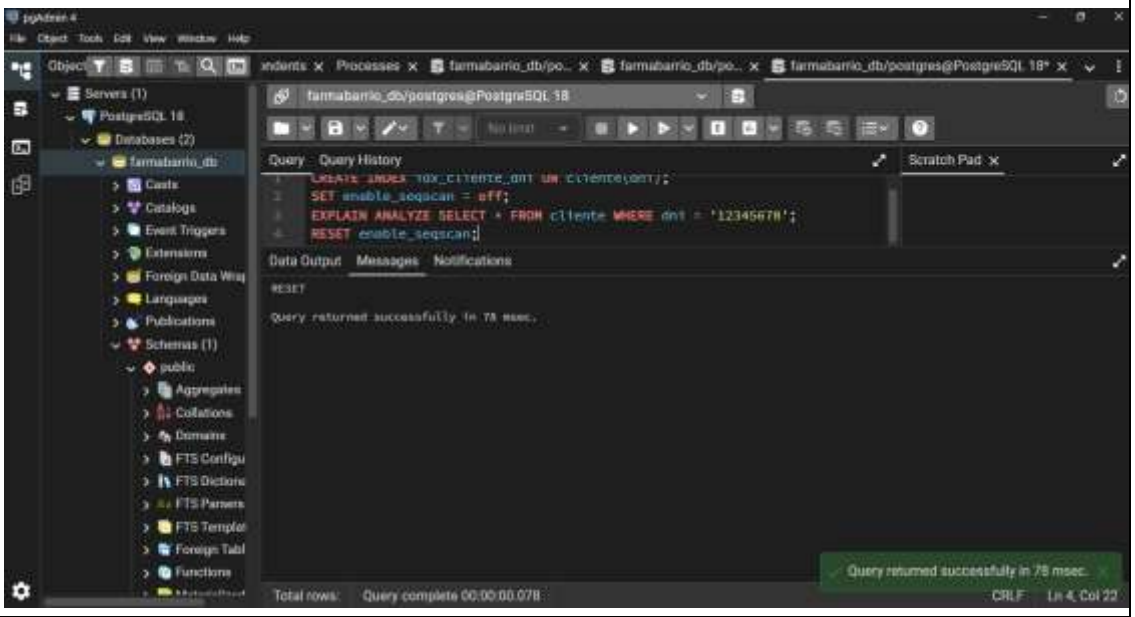


DEMOSTRACÓN (Comparen tiempos de consulta antes y después)

ANTES	DESPUES
Seq Scan on medicamento	Index Scan using idx_medicamento_nombre
Execution Time: mayor	Execution Time: menor

NOMBRE DEL TABLA - CAMPO: cliente (dni)	**AUTOR: MATENCIO PAUCAR, JORGE FABIAN**
CODIGO: CREATE INDEX idx_cliente_dni ON cliente(dni);	
JUSTIFICACION: La tabla cliente es consultada frecuentemente por el DNI para verificar la identidad del cliente en el momento de la venta o para aplicar descuentos de fidelidad. Un índice sobre el campo dni acelera estas consultas, permitiendo una atención más rápida en el punto de venta.	
EJECUCION:	

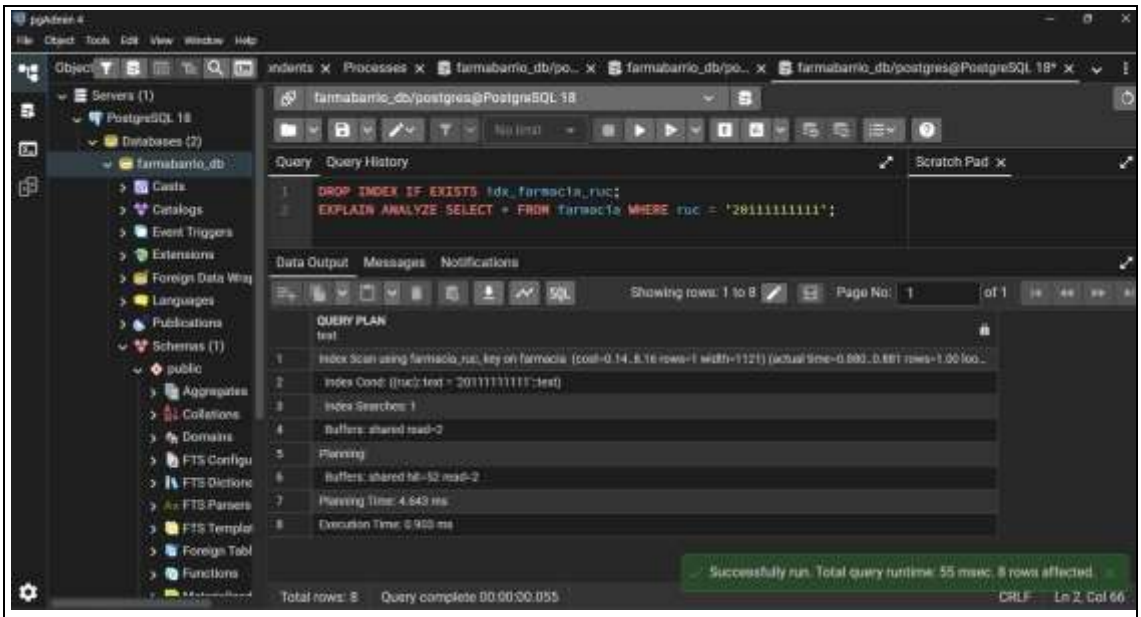


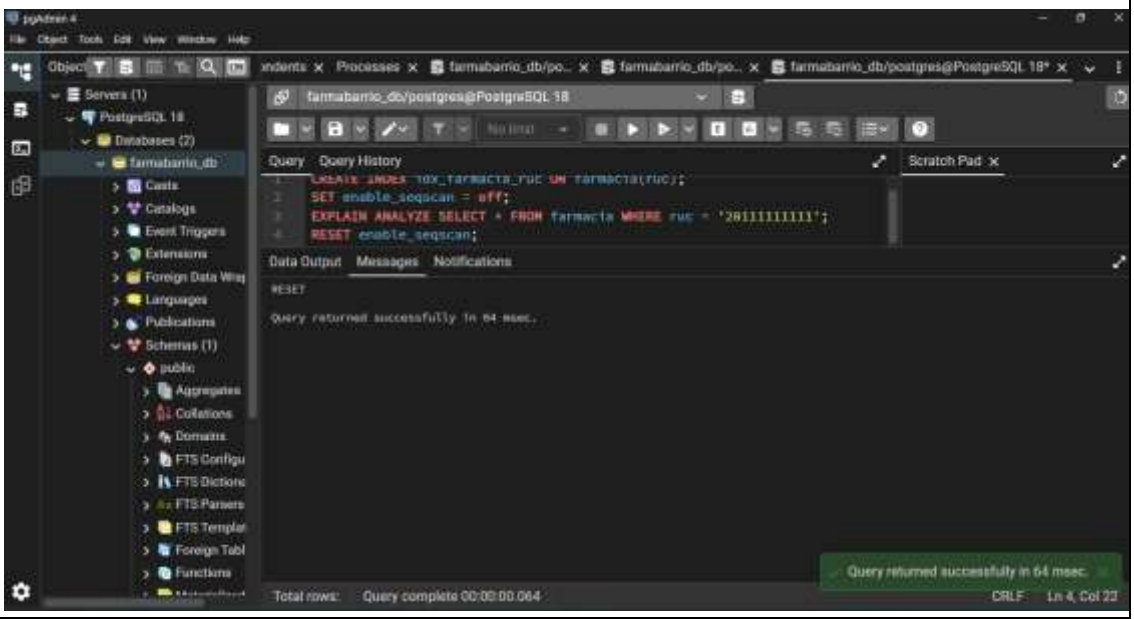


DEMOSTRACÓN (Comparen tiempos de consulta antes y después)

ANTES	DESPUES
Seq Scan on cliente	Index Scan using idx_cliente_dni
Execution Time: mayor	Execution Time: menor

NOMBRE DEL TABLA - CAMPO: farmacia (ruc)	**AUTOR: Rojas Pérez, Dereck**
CODIGO: CREATE INDEX idx_farmacia_ruc ON farmacia(ruc);	
JUSTIFICACION: La tabla farmacia es consultada por el RUC para validar la identidad de la farmacia en procesos administrativos, facturación y reportes. Un índice sobre el campo ruc permite que estas consultas sean más rápidas, mejorando el rendimiento en la gestión de la red de farmacias.	
EJECUCION:	





DEMOSTRACÓN (Comparen tiempos de consulta antes y después)

ANTES	DESPUES
Seq Scan on farmacia	Index Scan using idx_farmacia_ruc
Execution Time: mayor	Execution Time: menor

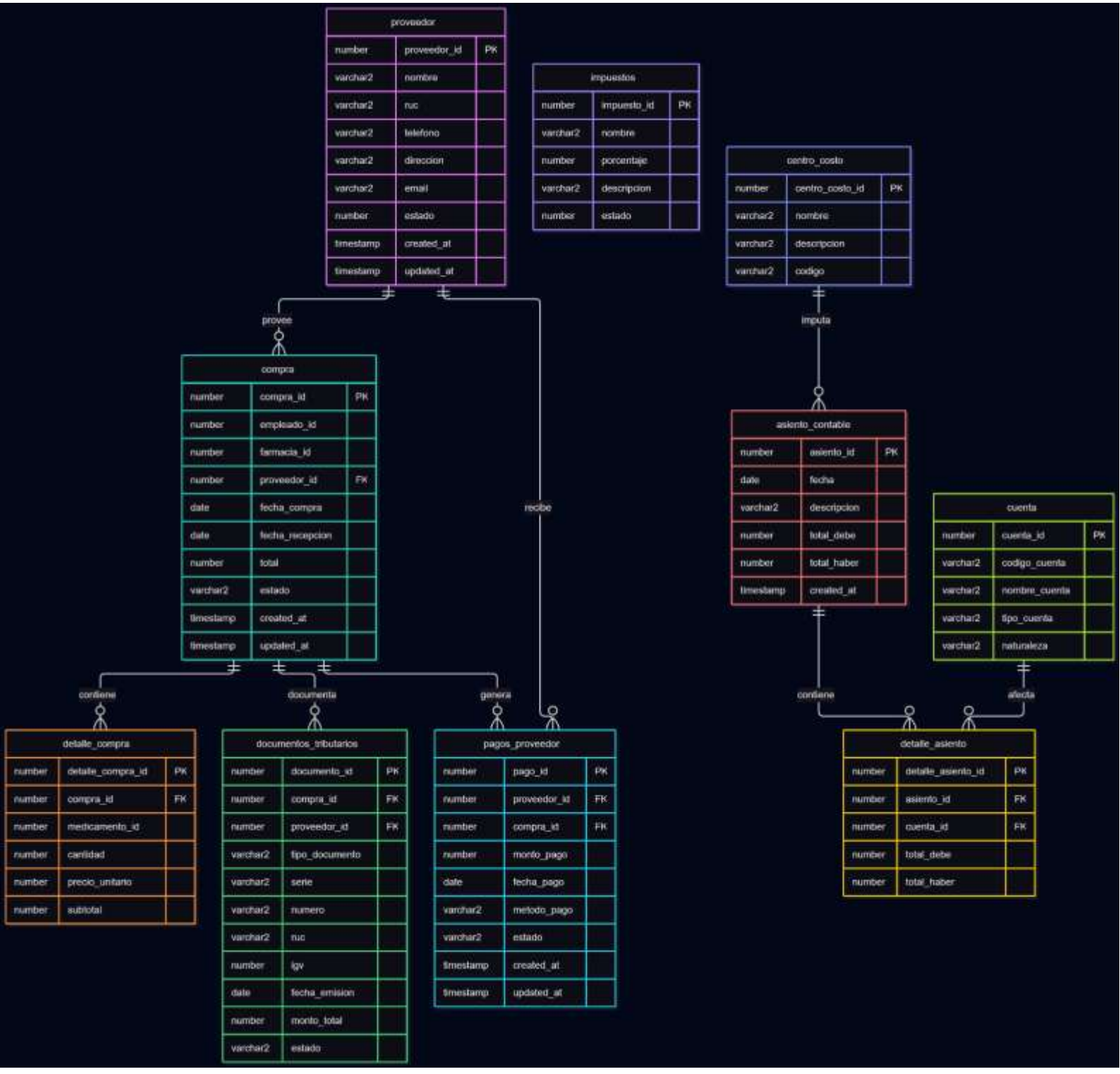
2.2.6 Comparativa de Sintaxis: PostgreSQL vs SQL Server

En la siguiente tabla comparativa se contrastan las principales diferencias sintácticas y de funciones entre el motor de base de datos PostgreSQL (empleado en FarmaBarrio) y Microsoft SQL Server:

Característica	PostgreSQL	SQL Server
Autoincremental	SERIAL / BIGSERIAL	IDENTITY(1,1)

Límite de resultados	LIMIT 10	TOP 10
Fecha actual	CURRENT_TIMESTAMP / CURRENT_DATE	GETDATE()
Concatenación	|| (ej. col1 || ' ' || col2)	+ (ej. col1 + ' ' + col2)
Manejo de nulos	COALESCE(campo, 0)	ISNULL(campo, 0) / COALESCE(campo, 0)
Paginación	LIMIT 5 OFFSET 10	OFFSET 10 ROWS FETCH NEXT 5 ROWS ONLY
Conversión de tipos (Casting)	campo::tipo o CAST(campo AS tipo)	CAST(campo AS tipo) o CONVERT(tipo, campo)

2.3 Base de datos: ORACLE



El modelo lógico de Oracle está compuesto por 10 tablas interrelacionadas que gestionan las compras, proveedores y finanzas de FarmaBarrio: proveedor, compra, detalle_compra, pagos_proveedor, documentos_tributarios, impuestos, asiento_contable, detalle_asiento, centro_costo y cuenta.

2.3.2 Tablas/Colecciones

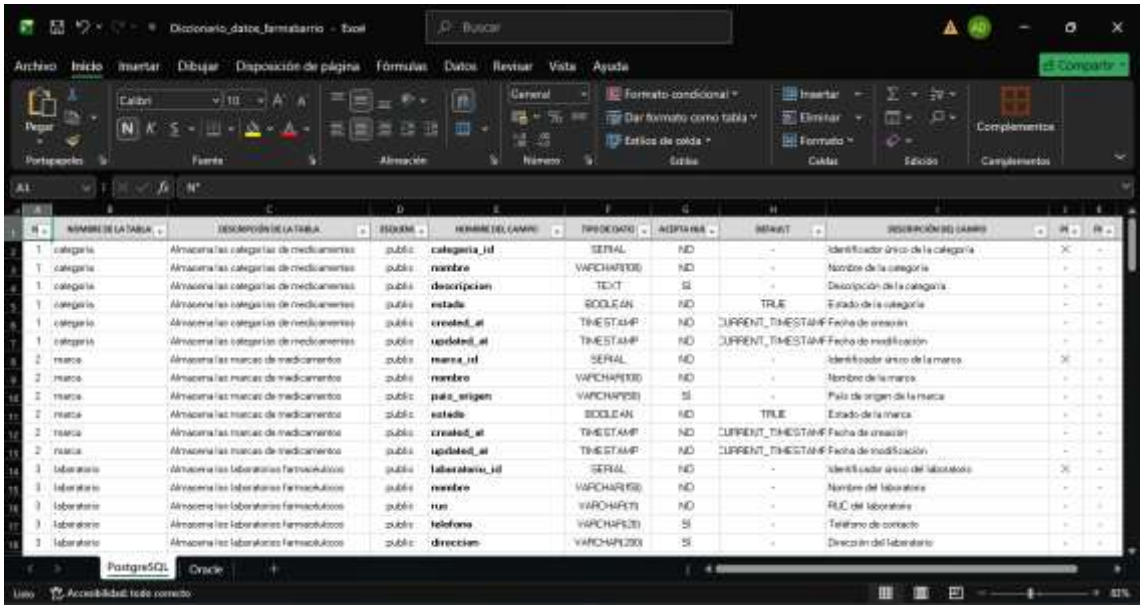
DESCRIPCIÓN DE TABLAS / COLECCIONES		
N°	**NOMBRE**	**DESCRIPCION**
1	proveedor	Proveedores de medicamentos con RUC, datos de contacto y estado
2	compra	Órdenes de compra realizadas a proveedores con fecha, total y estado
3	detalle_compra	Detalle de cada compra con medicamento, cantidad, precio unitario y subtotal
4	pagos_peveedor	Pagos realizados a proveedores con monto, fecha y método de pago
5	documentos_tributarios	Facturas, boletas y guías de remisión asociadas a las compras
6	impuestos	Tasas de impuestos aplicables (IGV, etc.) con porcentaje y descripción
7	asientos_contable	Asientos contables con fecha, descripción y totales debe/haber
8	detalle_asiento	Detalle de cada asiento contable con cuenta y montos debe/haber
9	dentro_costo	Centros de costo para imputación contable (por sucursal, área, etc.)
10	cuenta	Plan de cuentas contable con código, nombre, tipo y naturaleza

2.3.3 Diccionario de datos:

NOMBRE DE LA TABLA | ESQUEMA			**proveedor**			**farmabarrio_owner**		
DESCRIPCIÓN DE LA TABLA			Proveedores de medicamentos y suministros farmacéuticos de la red					
N°	**NOMBRE DEL CAMPO**	**TIPO DE DATO**	**ACEPTA NULO**	**DEFAULT**	**DESCRIPCIÓN DEL CAMPO**	**PK**	**FK**	**UK**
1	proveedor_id	NUMBER	NO	-	Identificador único del proveedor	**X**	-	-
2	nombre	VARCHAR2(150)	NO	-	Razón social / Nombre del proveedor	-	-	-
3	ruc	VARCHAR2(11)	NO	-	RUC del proveedor (11 dígitos)	-	-	**X**
4	telefono	VARCHAR2(20)	SÍ	-	Teléfono comercial de contacto	-	-	-
5	direccion	VARCHAR2(200)	SÍ	-	Dirección fiscal del proveedor	-	-	-
6	email	VARCHAR2(100)	SÍ	-	Correo electrónico comercial	-	-	-
7	estado	NUMBER(1)	NO	1	Estado del proveedor (1=Activo, 0=Inactivo)	-	-	-
8	created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	Fecha de registro en la plataforma	-	-	-

9	updated_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	Fecha de última modificación	-	-	-

El diccionario completo con las 10 tablas se encuentra en el archivo Excel adjunto: Diccionario_datos_farmabarrio.xlsx

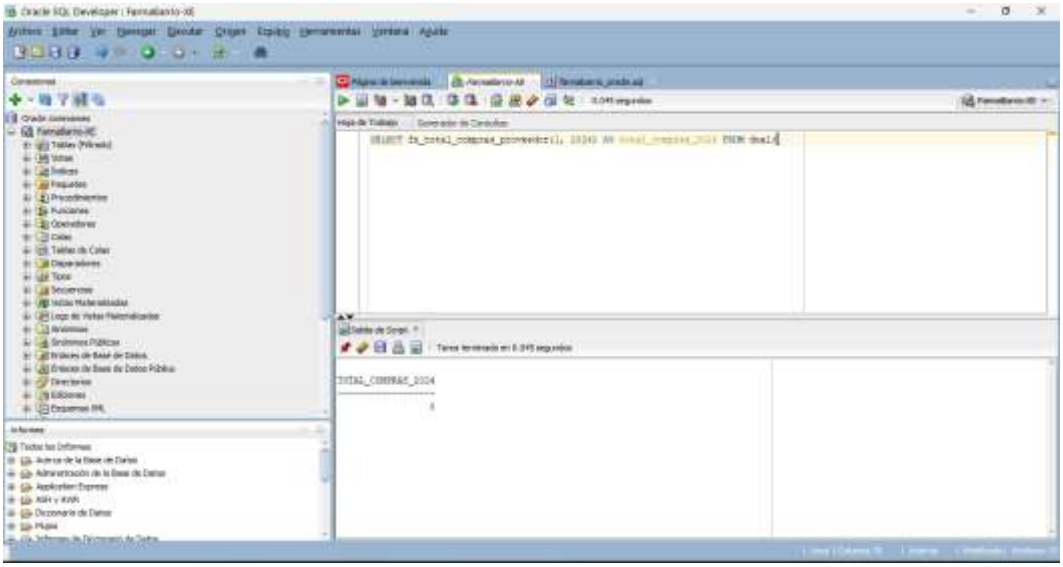


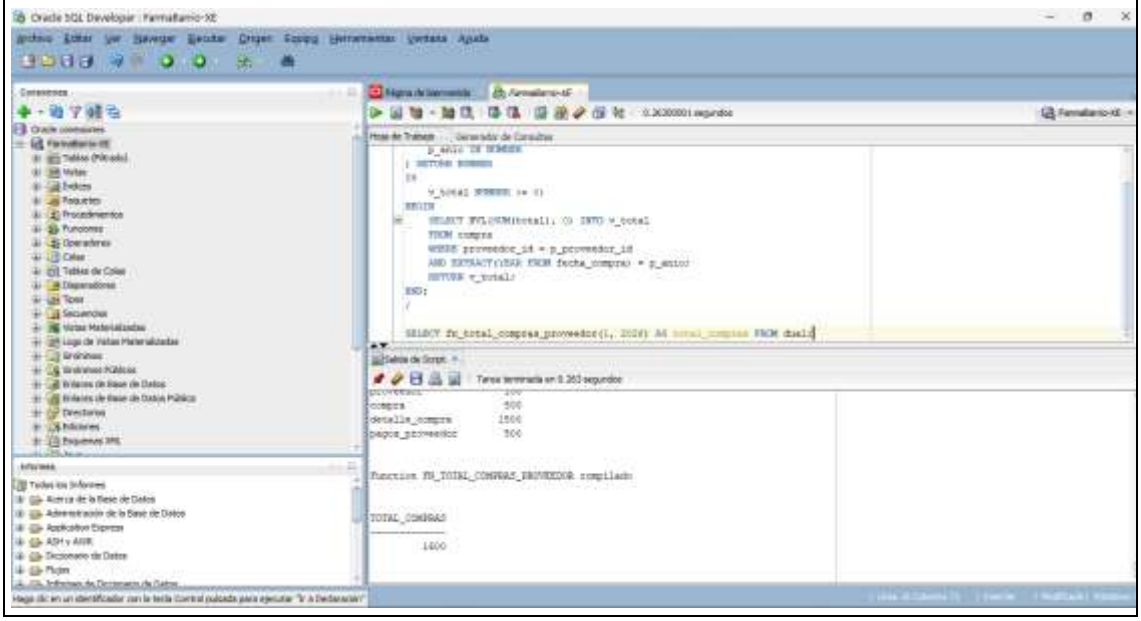
2.3.4 Procedimientos o funciones

NOMBRE: fn_total_compras_proveedor	**AUTOR: Grupo 2**
DESCRIPCION: Esta función calcula el total de compras realizadas a un proveedor en un año fiscal específico. Recibe como parámetros el ID del proveedor y el año. Retorna el monto total de las compras realizadas.	
CODIGO DEL PROCEDIMEINTO O FUNCION **CREATE OR REPLACE FUNCTION fn_total_compras_proveedor(** **p_proveedor_id IN NUMBER,** **p_anio IN NUMBER** **) RETURN NUMBER** **IS** **v_total NUMBER := 0;** **BEGIN** **SELECT NVL(SUM(total), 0) INTO v_total** **FROM compra** **WHERE proveedor_id = p_proveedor_id** **AND EXTRACT(YEAR FROM fecha_compra) = p_anio;** **RETURN v_total;** **END;** **/**	
EJECUCION DEL PROCEDIMIENTO O FUNCION *SELECT fn_total_compras_proveedor(1, 2026) AS total_compras FROM dual;*	

RESULTADO – CAPTURA

SELECT fn_total_compras_proveedor(1, 2026) AS total_compras FROM dual;





NOMBRE: sp_actualizar_compras_vencidas	AUTOR: QUIÑONES DIEGO, MAYKEL PIERO (Líder)

DESCRIPCION: Este procedimiento automatiza la anulación de órdenes de compra pendientes que han expirado. Busca todas las compras en estado 'PENDIENTE' cuya fecha de compra supere los 90 días de antigüedad y actualiza su estado a 'ANULADA', informando la cantidad de registros procesados.

CODIGO DEL PROCEDIMIENTO O FUNCION:

```
CREATE OR REPLACE PROCEDURE sp_actualizar_compras_vencidas
IS
    v_count NUMBER := 0;
BEGIN
    UPDATE compra
    SET estado = 'ANULADA',
        updated_at = SYSTIMESTAMP
    WHERE estado = 'PENDIENTE'
      AND fecha_compra < SYSDATE - 90;

    v_count := SQL%ROWCOUNT;
```

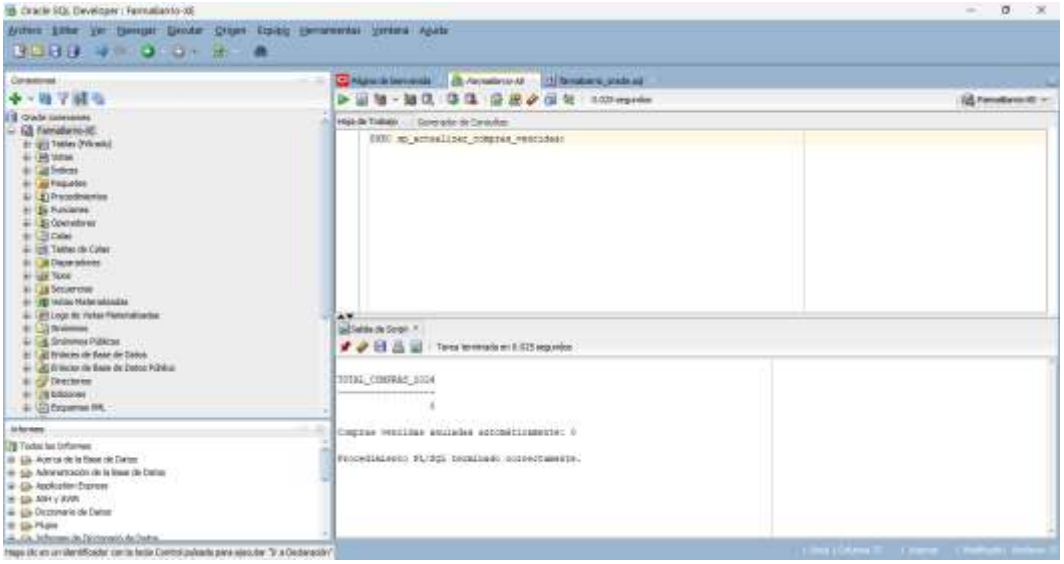
```
    DBMS_OUTPUT.PUT_LINE('Compras anuladas por vencimiento: ' || v_count);
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error en sp_actualizar_compras_vencidas: ' || SQLERRM);
END sp_actualizar_compras_vencidas;
/
```

EJECUCION DEL PROCEDIMIENTO O FUNCION:

```
EXEC sp_actualizar_compras_vencidas;
```

RESULTADO – CAPTURA

EXEC sp_actualizar_compras_vencidas;



NOMBRE: fn_promedio_compras_mes	**AUTOR: GARCIA ESPEJO, NICOLE ANDREA**

DESCRIPCION: Esta función calcula y retorna el monto promedio de las compras realizadas a proveedores durante un mes y año específicos. Considera únicamente compras activas y válidas (excluyendo transacciones en estado 'ANULADA'). Si no se registran compras en dicho periodo, retorna 0.00.

CODIGO DEL PROCEDIMIENTO O FUNCION:

```
CREATE OR REPLACE FUNCTION fn_promedio_compras_mes(
    p_mes  IN NUMBER,
    p_anio IN NUMBER
) RETURN NUMBER
IS
    v_promedio NUMBER(10,2) := 0;
BEGIN
    SELECT NVL(AVG(total), 0)
    INTO v_promedio
    FROM compra
    WHERE EXTRACT(MONTH FROM fecha_compra) = p_mes
      AND EXTRACT(YEAR FROM fecha_compra) = p_anio
      AND estado != 'ANULADA';

    RETURN ROUND(v_promedio, 2);
EXCEPTION
    WHEN OTHERS THEN
        RETURN 0;
END fn_promedio_compras_mes;
```

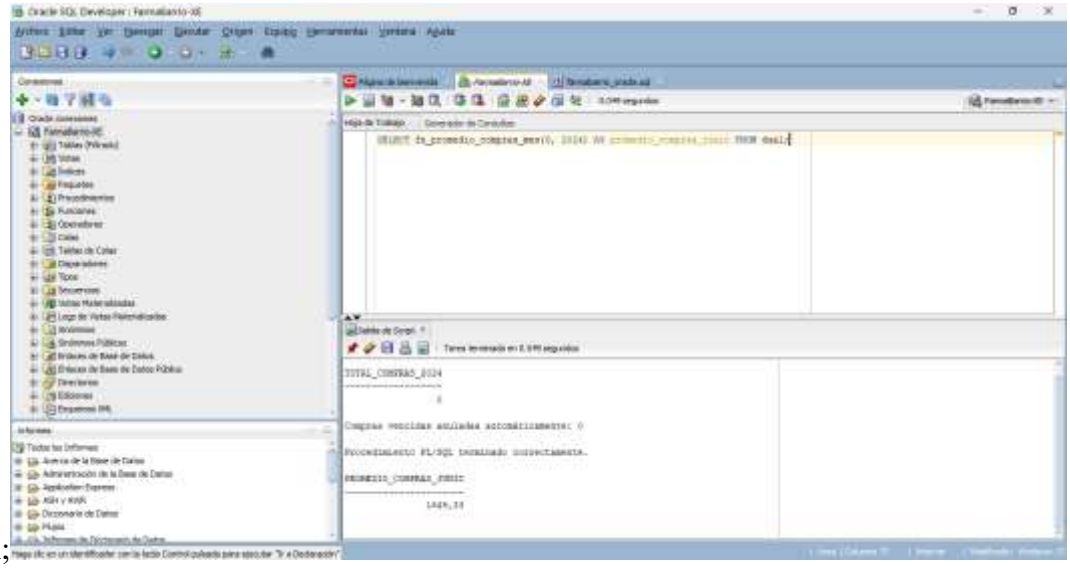
```
/
```

EJECUCION DEL PROCEDIMIENTO O FUNCION:

```
SELECT fn_promedio_compras_mes(3, 2026) AS promedio_compras_marzo FROM dual;
```

RESULTADO – CAPTURA

SELECT fn_promedio_compras_mes(3, 2026) AS promedio_compras_marzo FROM dual;



NOMBRE: sp_registrar_pago_proveedor	AUTOR: MATENCIO PAUCAR, JORGE FABIAN

DESCRIPCION: Este procedimiento registra un nuevo abono o pago a un proveedor asociado a una orden de compra. Valida que el monto sea mayor a 0, inserta el pago en la tabla pagos_proveedor con fecha actual, verifica si la suma de pagos salda el total de la compra y, en tal caso, actualiza su estado a 'PAGADA'.

CODIGO DEL PROCEDIMIENTO O FUNCION:

```
CREATE OR REPLACE PROCEDURE sp_registrar_pago_proveedor(
    p_proveedor_id IN NUMBER,
    p_compra_id    IN NUMBER,
    p_monto        IN NUMBER,
    p_metodo_pago  IN VARCHAR2
)
IS
    v_nuevo_id      NUMBER;
    v_total_compra  NUMBER(10,2);
    v_total_pagado  NUMBER(10,2);
BEGIN
    IF p_monto <= 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'El monto del pago debe ser estrictamente mayor a 0');
    END IF;

    SELECT NVL(MAX(pago_id), 0) + 1 INTO v_nuevo_id FROM pagos_proveedor;

    INSERT INTO pagos_proveedor (
        pago_id, proveedor_id, compra_id, monto_pago,
        fecha_pago, metodo_pago, estado, created_at, updated_at
    ) VALUES (
        v_nuevo_id, p_proveedor_id, p_compra_id, p_monto,
        SYSDATE, p_metodo_pago, 'PAGADO', SYSTIMESTAMP, SYSTIMESTAMP
    );

    SELECT total INTO v_total_compra FROM compra WHERE compra_id = p_compra_id;
```

```
    SELECT NVL(SUM(monto_pago), 0) INTO v_total_pagado
    FROM pagos_proveedor
    WHERE compra_id = p_compra_id AND estado = 'PAGADO';

    IF v_total_pagado >= v_total_compra THEN
        UPDATE compra SET estado = 'PAGADA', updated_at = SYSTIMESTAMP WHERE compra_id = p_compra_id;
    END IF;

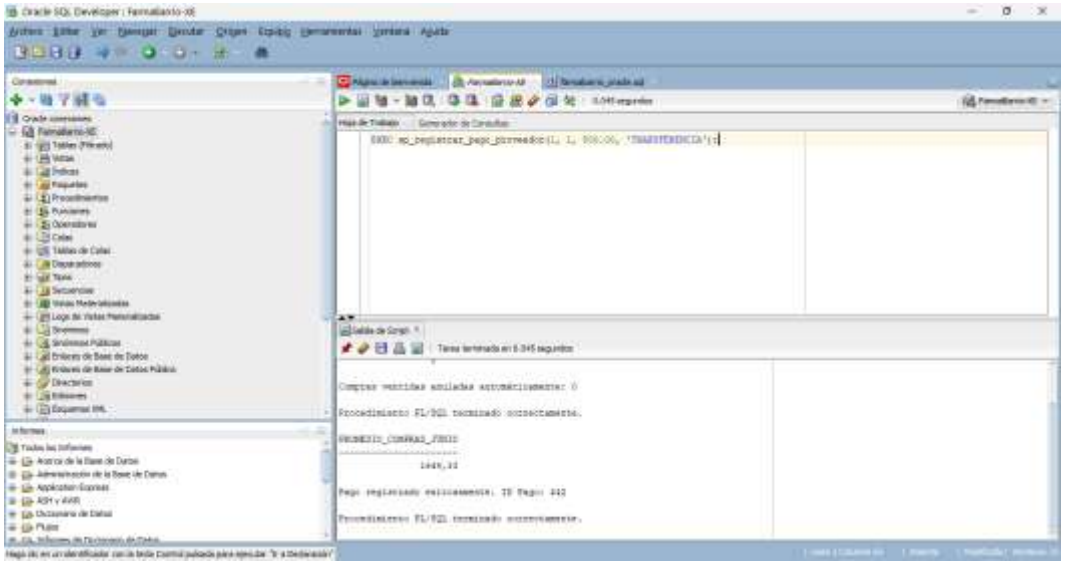
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Pago registrado exitosamente con ID: ' || v_nuevo_id);
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20002, 'Error al registrar pago: ' || SQLERRM);
END sp_registrar_pago_proveedor;
/
```

EJECUCION DEL PROCEDIMIENTO O FUNCION:

```
EXEC sp_registrar_pago_proveedor(1, 1, 1500.00, 'TRANSFERENCIA');
```

RESULTADO – CAPTURA

EXEC sp_registrar_pago_proveedor(1, 1, 1500.00, 'TRANSFERENCIA');



NOMBRE: fn_saldo_pendiente_proveedor	AUTOR: ROJAS PÉREZ, DERECK

DESCRIPCION: Esta función calcula el saldo pendiente total de pago a un proveedor específico. Obtiene la sumatoria total de las compras emitidas al proveedor y resta el acumulado de pagos registrados en pagos_proveedor. Si no adeuda, retorna 0.00.

CODIGO DEL PROCEDIMIENTO O FUNCION:

```
CREATE OR REPLACE FUNCTION fn_saldo_pendiente_proveedor(
    p_proveedor_id IN NUMBER
) RETURN NUMBER
IS
    v_total_compras NUMBER(10,2) := 0;
    v_total_pagos   NUMBER(10,2) := 0;
    v_saldo         NUMBER(10,2) := 0;
BEGIN
    SELECT NVL(SUM(total), 0)
    INTO v_total_compras
    FROM compra
```

```
    WHERE proveedor_id = p_proveedor_id
      AND estado != 'ANULADA';

    SELECT NVL(SUM(monto_pago), 0)
    INTO v_total_pagos
    FROM pagos_proveedor
    WHERE proveedor_id = p_proveedor_id
      AND estado = 'PAGADO';

    v_saldo := v_total_compras - v_total_pagos;

    IF v_saldo < 0 THEN
        v_saldo := 0;
    END IF;

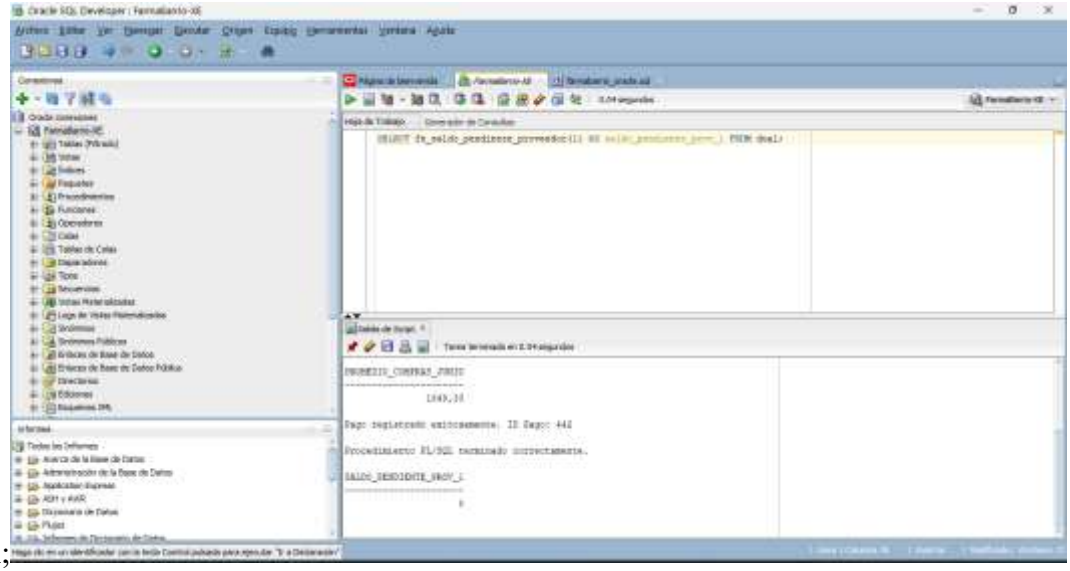
    RETURN ROUND(v_saldo, 2);
EXCEPTION
    WHEN OTHERS THEN
        RETURN 0;
END fn_saldo_pendiente_proveedor;
/
```

EJECUCION DEL PROCEDIMIENTO O FUNCION:

```
SELECT fn_saldo_pendiente_proveedor(1) AS saldo_pendiente_proveedor_1 FROM dual;
```

RESULTADO – CAPTURA

SELECT fn_saldo_pendiente_proveedor(1) AS saldo_pendiente_proveedor_1 FROM dual;



2.3.5 Indices

NOMBRE DEL TABLA - CAMPO: compra (proveedor_id, estado)	AUTOR: QUIÑONES DIEGO, MAYKEL PIERO

CODIGO:

```
CREATE INDEX idx_compra_proveedor_estado ON compra(proveedor_id, estado);
```

JUSTIFICACION:

La tabla compra es consultada frecuentemente por proveedor_id (para obtener el histórico de un proveedor específico) y por estado (para listar compras pendientes, completadas, etc.). Un índice compuesto por ambos campos permite que las consultas que filtran por estos dos criterios accedan directamente a los registros sin necesidad de recorrer toda la tabla.

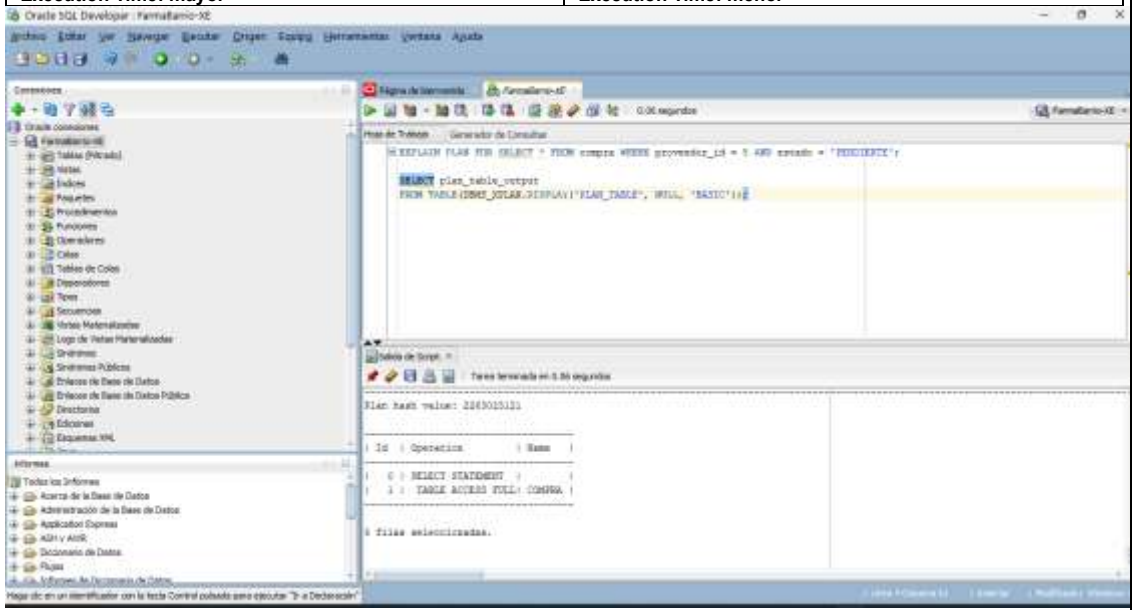
EJECUCION:

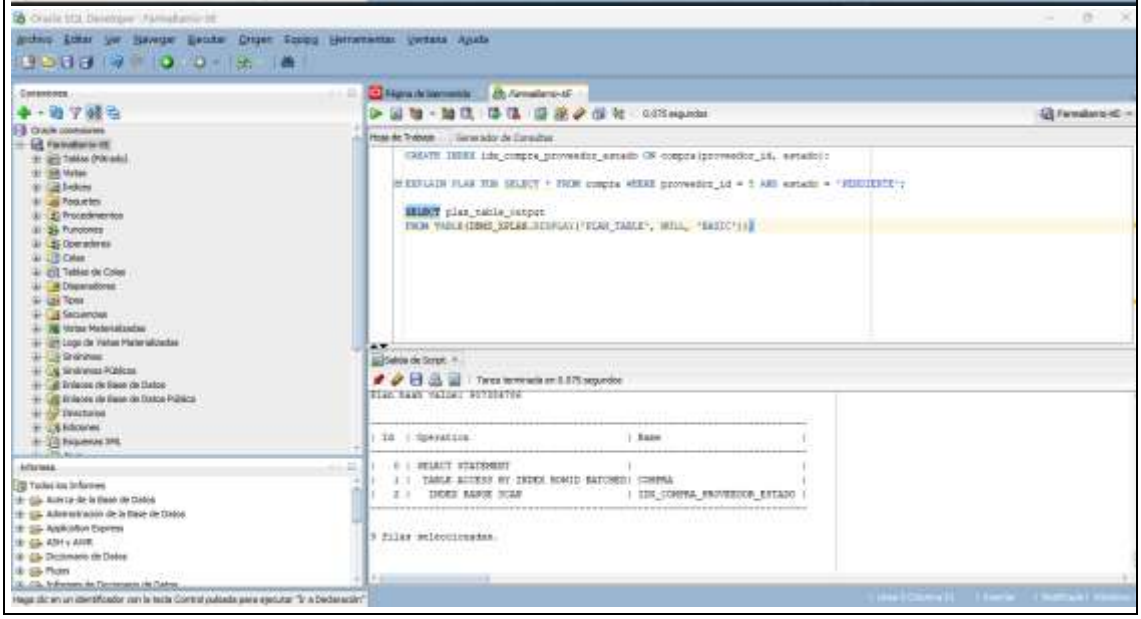
```
CREATE INDEX idx_compra_proveedor_estado ON compra(proveedor_id, estado);

EXPLAIN PLAN FOR SELECT * FROM compra WHERE proveedor_id = 5 AND estado = 'PENDIENTE';
SELECT plan_table_output FROM TABLE(DBMS_XPLAN.DISPLAY('PLAN_TABLE', NULL, 'BASIC'));
```

DEMOSTRACÓN (Comparen tiempos de consulta antes y después)

ANTES	DESPUES
TABLE ACCESS FULL	INDEX RANGE SCAN
Execution Time: mayor	Execution Time: menor





NOMBRE DEL TABLA - CAMPO: compra (fecha_compra)	**AUTOR: WONG VILLANUEVA, LEONARDO JOSUÉ**

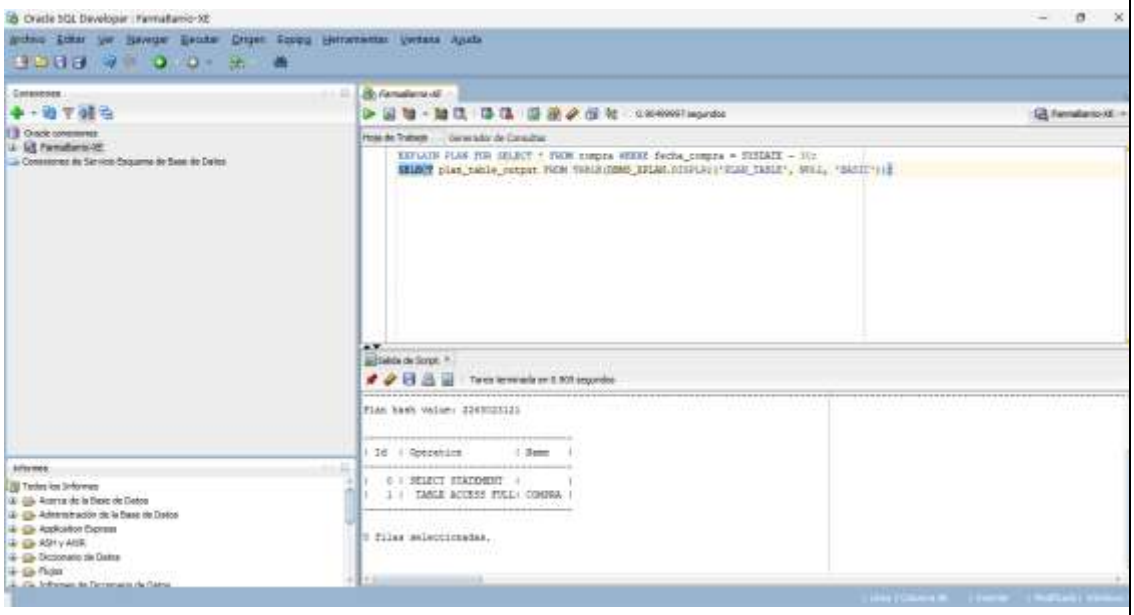
CODIGO: CREATE INDEX idx_compra_fecha ON compra(fecha_compra);

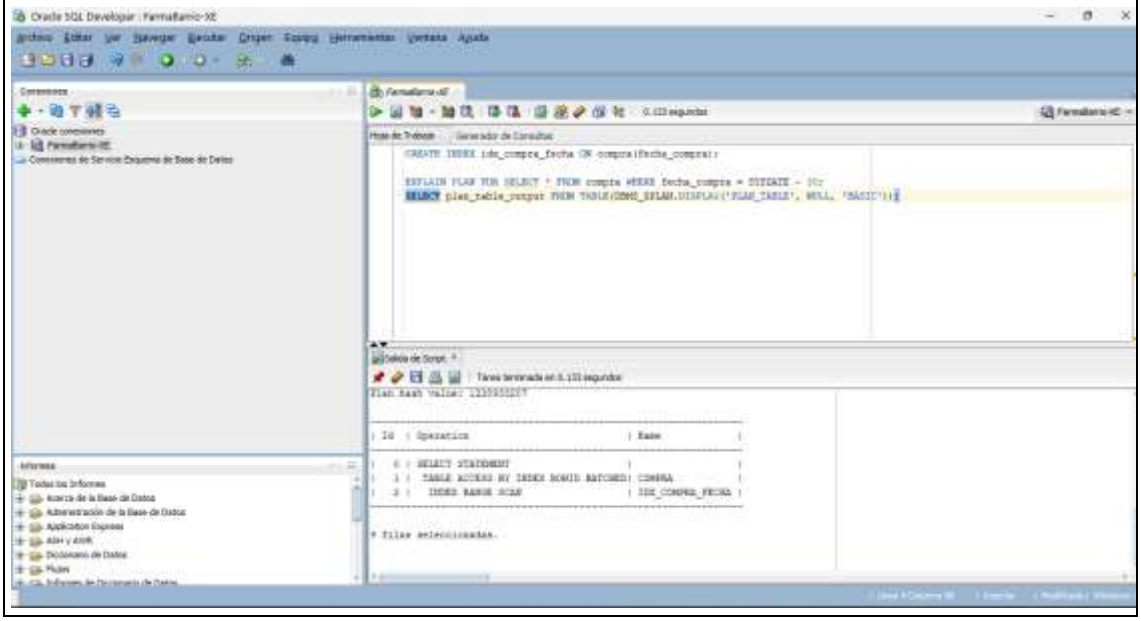
JUSTIFICACION: La tabla compra almacena todas las órdenes de compra. Las consultas más frecuentes filtran por rango de fechas para generar reportes de compras por período. Un índice sobre el campo fecha_compra acelera este tipo de consultas.

EJECUCION: CREATE INDEX idx_compra_fecha ON compra(fecha_compra);

DEMOSTRACÓN (Comparen tiempos de consulta antes y después)

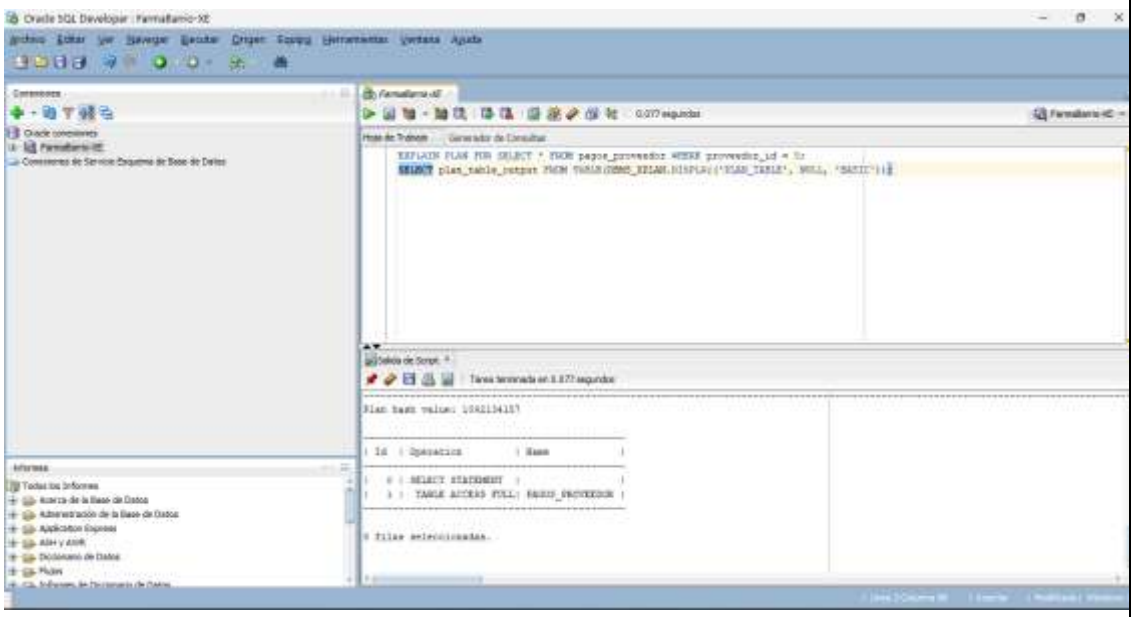
ANTES	DESPUES
TABLE ACCESS FULL	INDEX RANGE SCAN
Execution Time: mayor	Execution Time: menor

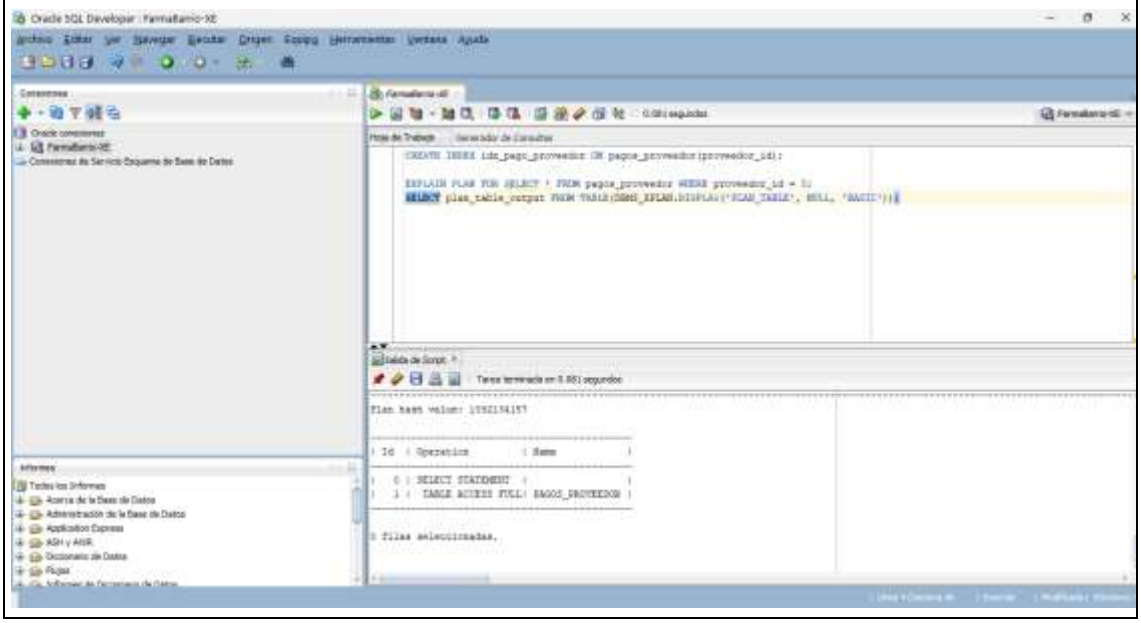




NOMBRE DEL TABLA - CAMPO: pagos_proveedor (proveedor_id)	AUTOR: GARCIA ESPEJO, NICOLE ANDREA
CODIGO: CREATE INDEX idx_pago_proveedor ON pagos_proveedor(proveedor_id);	
JUSTIFICACION: La tabla pagos_proveedor es consultada frecuentemente por proveedor_id para obtener el historial de pagos realizados a un proveedor específico. Un índice sobre este campo acelera las consultas de conciliación de pagos.	
EJECUCION: CREATE INDEX idx_pago_proveedor ON pagos_proveedor(proveedor_id);	
DEMOSTRACÓN (Comparen tiempos de consulta antes y después)	

ANTES	DESPUES





NOMBRE DEL TABLA - CAMPO: documentos_tributarios (ruc)	AUTOR: MATENCIO PAUCAR, JORGE FABIAN

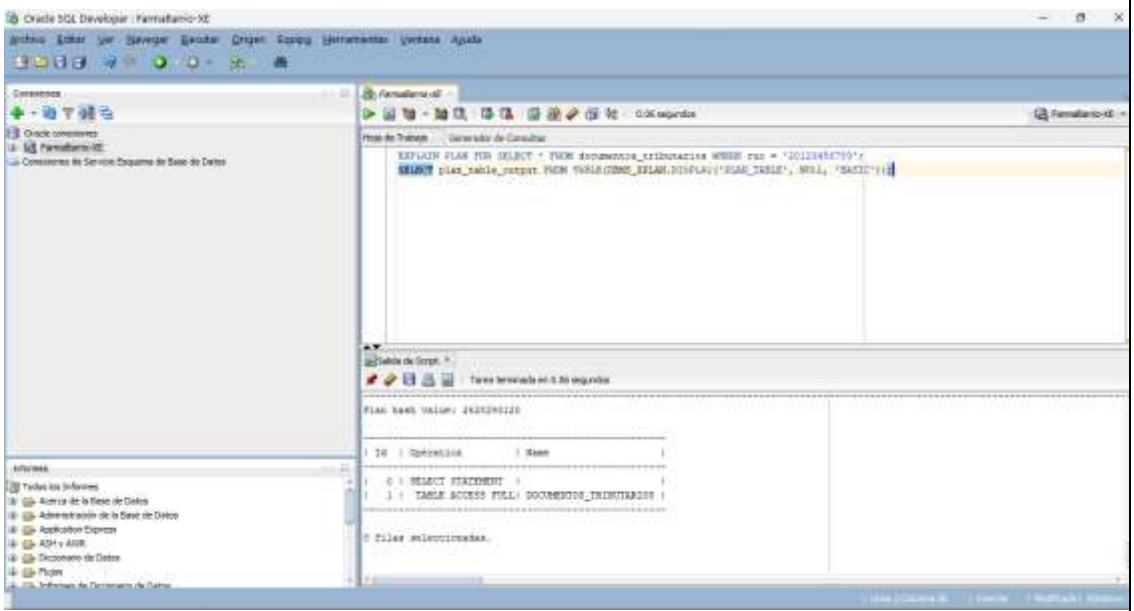
CODIGO: CREATE INDEX idx_documento_ruc ON documentos_tributarios(ruc);

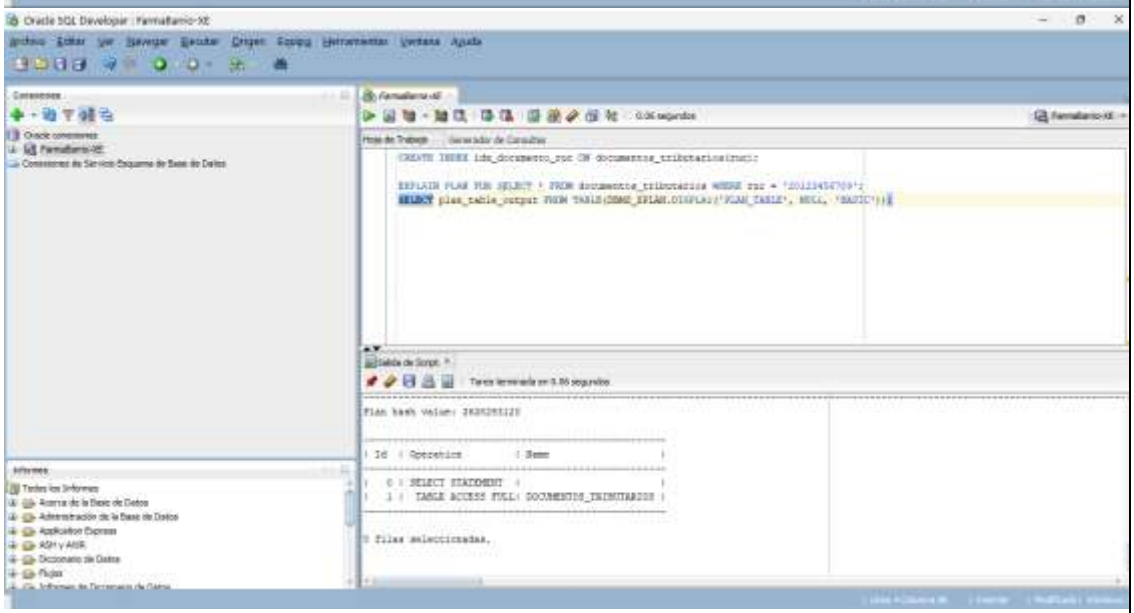
JUSTIFICACION:
La tabla documentos_tributarios es consultada por RUC para validar la identidad del proveedor en procesos de facturación y auditoría tributaria. Un índice sobre este campo acelera estas consultas.

EJECUCION:
CREATE INDEX idx_documento_ruc ON documentos_tributarios(ruc);

DEMOSTRACÓN (Comparen tiempos de consulta antes y después)

ANTES	DESPUES
TABLE ACCESS FULL	INDEX RANGE SCAN
Execution Time: mayor	Execution Time: menor





NOMBRE DEL TABLA - CAMPO: proveedor (ruc)	AUTOR :ROJAS PÉREZ, DERECK

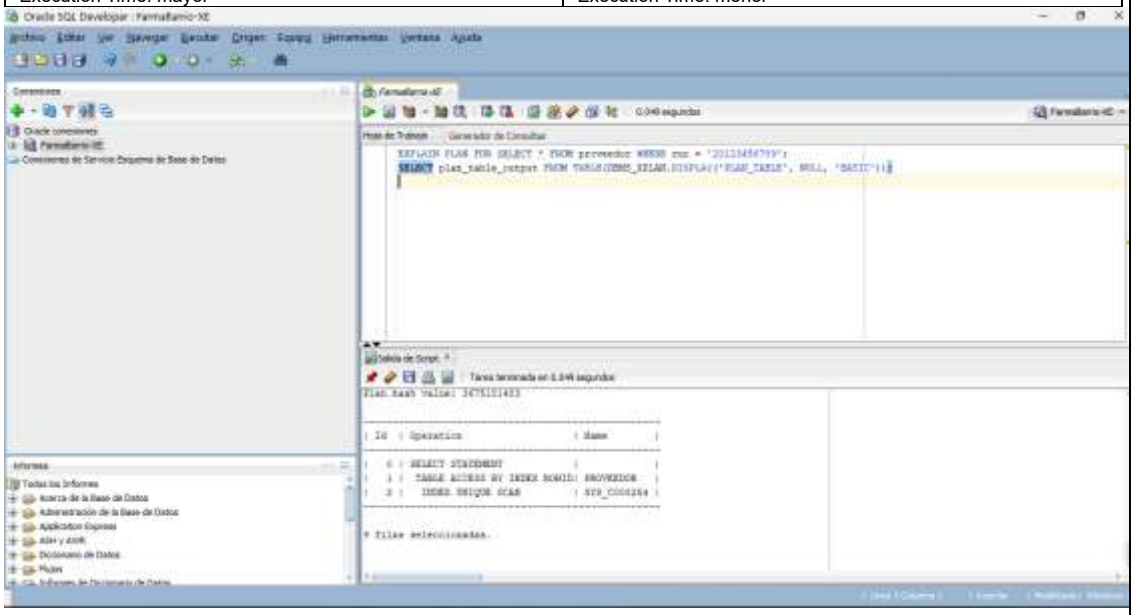
CODIGO:
CREATE INDEX idx_proveedor_ruc ON proveedor(ruc);

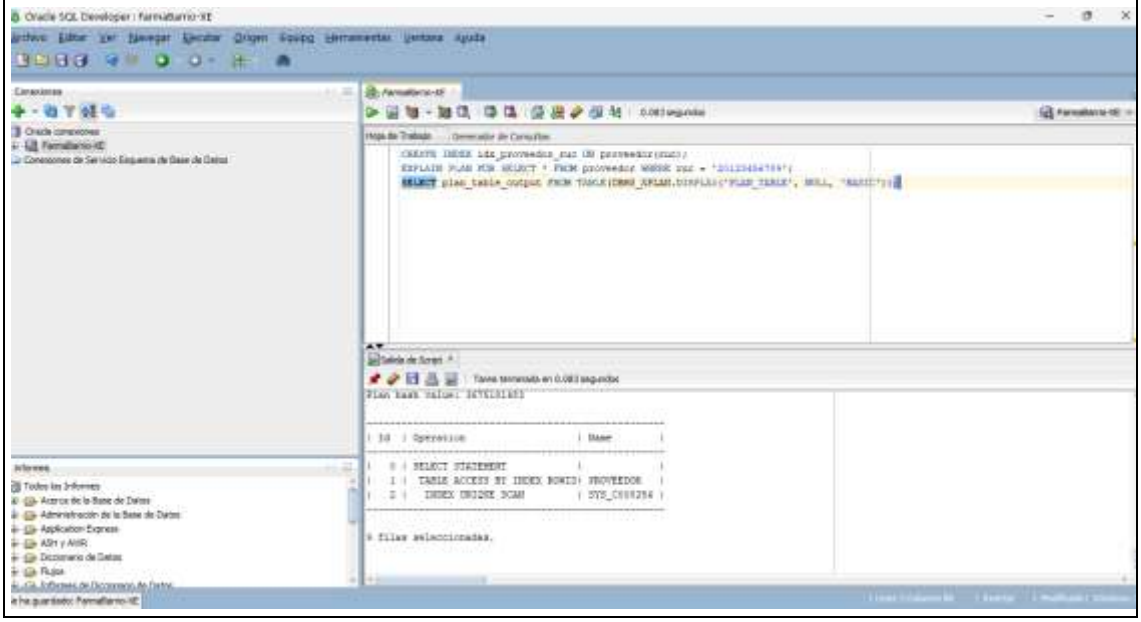
JUSTIFICACION:
La tabla proveedor es consultada por RUC para validar la identidad del proveedor en procesos administrativos y de facturación. Un índice sobre este campo permite que estas consultas sean más rápidas.

EJECUCION: CREATE INDEX idx_proveedor_ruc ON proveedor(ruc);

DEMOSTRACÓN (Comparen tiempos de consulta antes y después)

ANTES	DESPUES
TABLE ACCESS FULL	INDEX RANGE SCAN
Execution Time: mayor	Execution Time: menor





2.3.6 Comparativa de Sintaxis: Oracle vs SQL Server

En la siguiente tabla comparativa se contrastan las principales diferencias sintácticas y arquitectónicas entre Oracle Database y Microsoft SQL Server:

Característica	Oracle	SQL Server
Autoincremental	GENERATED BY DEFAULT AS IDENTITY o Secuencia + Trigger	IDENTITY(1,1)
Límite de resultados	FETCH FIRST 10 ROWS ONLY o WHERE ROWNUM <= 10	TOP 10
Fecha actual	SYSDATE / SYSTIMESTAMP	GETDATE() / CURRENT_TIMESTAMP
Formato de fecha	TO_CHAR(SYSDATE, 'DD/MM/YYYY')	CONVERT(VARCHAR, GETDATE(), 103)
Concatenación	|| (ej. col1 || ' ' || col2)	+ (ej. col1 + ' ' + col2)
Manejo de nulos	NVL(campo, 0) / COALESCE(campo, 0)	ISNULL(campo, 0) / COALESCE(campo, 0)
Paginación	OFFSET 10 ROWS FETCH NEXT 5 ROWS ONLY	OFFSET 10 ROWS FETCH NEXT 5 ROWS ONLY
Conversión de tipos (Casting)	CAST(campo AS tipo) / TO_NUMBER() / TO_CHAR()	CAST(campo AS tipo) / CONVERT(tipo, campo)
Tabla Dual	FROM dual (obligatorio para expresiones sin tabla)	No requerida (ej. SELECT 1 + 1)

2.4 Base de datos: MongoDB

2.5 Base de datos: Cassandra

2.6 Base de datos: Hadoop

3 DISEÑO DE ESQUEMA DE FRAGMENTACIÓN

(Implementar al menos en una tabla en PostgreSQL, Oracle y MongoDB)

3.1 PostgreSQL

3.1.1 Criterio de Fragmentación:

(Expliquen qué campo usan para fragmentar los datos y por qué)

3.1.2 Definición de Fragmentos:

(Describan cada fragmento con su condición de pertenencia)

Fragmento	Criterio de pertenencia
F1	Ejemplo: región = 'Norte'
F2	Ejemplo: región = 'Sur'

3.1.3 Diagrama de Distribución de Fragmentos:

(Inserten aquí un esquema que muestre la fragmentación)

3.2 Oracle

3.3 MongoDB

4 IMPLEMENTACIÓN DE REPLICACIÓN ASINCRÓNICA

4.1 Descripción de la Configuración de Replicación:

- Base de datos seleccionada: MongoDB / Cassandra
- Número de nodos creados: ___
- Tipo de replicación configurada: ___

4.2 Capturas de Pantalla:

(Adjunten capturas que muestren la configuración y la replicación funcionando)

5 CONSULTAS DIFERENTES EN NOSQL

Cada integrante debe realizar 1 consultas distintas. Cada consulta debe responder al objetivo del proyecto

AUTOR:	BASE DE DATOS:
CODIGO:	
JUSTIFICACION:	
EJECUCION:	
RESULTADO (CAPTURA):	

AUTOR:	BASE DE DATOS:
CODIGO:	
JUSTIFICACION:	
EJECUCION:	
RESULTADO (CAPTURA):	

6 ADMINISTRACIÓN DE USUARIOS

Cada integrante debe tener al menos un usuario teniendo diferentes privilegios como amerita los requerimientos del proyecto, debe haber un usuario al menos por cada base de datos.

6.1 Administración de Usuarios: PostgreSQL

USUARIO: farma_dba / farma_app / farma_readonly	BASE DE DATOS: PostgreSQL (farmabarrio_db)

ROLES CREADOS:

• farma_dba: Rol de administración total (SUPERUSER, CREATEDB, CREATEROLE).
• farma_app: Rol operativo para el backend de la aplicación (DML sobre esquema public).
• farma_readonly: Rol de consulta para analítica y reportería (SELECT exclusivo).

CODIGO CREACIÓN DE USUARIOS / ROLES:

```
-- 1. Administrador (DBA)
CREATE ROLE farma_dba WITH LOGIN PASSWORD 'Dba_2026!' SUPERUSER CREATEROLE CREATEDB REPLICATION;

-- 2. Usuario de Aplicación (Backend CRUD)
CREATE ROLE farma_app WITH LOGIN PASSWORD 'App_2026!' NOSUPERUSER NOCREATEDB NOCREATEROLE;

-- 3. Usuario de Solo Lectura (Auditoría y Reportes)
CREATE ROLE farma_readonly WITH LOGIN PASSWORD 'Read_2026!' NOSUPERUSER NOCREATEDB NOCREATEROLE;
```

CODIGO ASIGNACION DE PRIVILEGIOS:

```
-- Conexión a la base de datos
GRANT CONNECT ON DATABASE farmabarrio_db TO farma_dba, farma_app, farma_readonly;

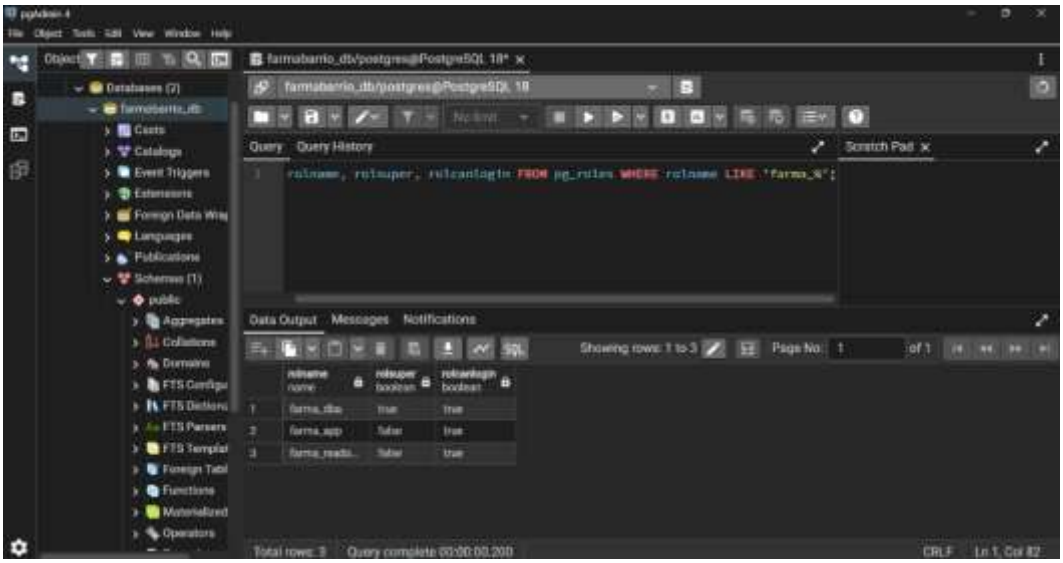
-- Privilegios farma_app (DML completo sobre tablas y secuencias)
GRANT USAGE ON SCHEMA public TO farma_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO farma_app;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO farma_app;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO
farma_app;

-- Privilegios farma_readonly (Solo lectura)
GRANT USAGE ON SCHEMA public TO farma_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO farma_readonly;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO farma_readonly;
```

DESCRIPCION DE PRIVILEGIOS OTORGADOS:

Se implementa el principio de mínimo privilegio: farma_dba cuenta con control administrativo total para tareas de mantenimiento y DDL; farma_app cuenta exclusivamente con permisos DML (SELECT, INSERT, UPDATE, DELETE) y uso de secuencias seriales en el esquema public, garantizando que el aplicativo no pueda alterar la estructura de tablas; farma_readonly dispone de acceso restringido a sentencias SELECT para reportería segura sin riesgo de alteración de datos.

CAPTURA DE PANTALLA: (Evidencia de creación de usuarios y roles en PostgreSQL)



6.2 Administración de Usuarios: Oracle

USUARIO: farma_dba / farma_app / farma_readonly	**BASE DE DATOS: Oracle (PDB / SYSTEM)**

ROLES Y PERFILES ASIGNADOS:

• farma_dba: Perfil administrativo con roles CONNECT, RESOURCE y DBA con cuota ilimitada en tablespace.
• farma_app: Usuario de servicio con rol CREATE SESSION y cuota 500M en ts_farmabarrio para transacciones de compras.
• farma_readonly: Usuario de consulta con rol CREATE SESSION y cuota 0 para generación de estados contables.

CODIGO CREACIÓN DE USUARIOS:

```
-- 1. Administrador (DBA)
CREATE USER farma_dba IDENTIFIED BY "Dba_2026!"
DEFAULT TABLESPACE users TEMPORARY TABLESPACE temp
QUOTA UNLIMITED ON users PROFILE DEFAULT ACCOUNT UNLOCK;

-- 2. Usuario de Aplicación
CREATE USER farma_app IDENTIFIED BY "App_2026!"
DEFAULT TABLESPACE users TEMPORARY TABLESPACE temp
QUOTA 500M ON users PROFILE DEFAULT ACCOUNT UNLOCK;

-- 3. Usuario de Solo Lectura
CREATE USER farma_readonly IDENTIFIED BY "Read_2026!"
DEFAULT TABLESPACE users TEMPORARY TABLESPACE temp
QUOTA 0 ON users PROFILE DEFAULT ACCOUNT UNLOCK;
```

CODIGO ASIGNACION DE PRIVILEGIOS:

```
-- Privilegios farma_dba
GRANT CREATE SESSION, CONNECT, RESOURCE, DBA TO farma_dba;

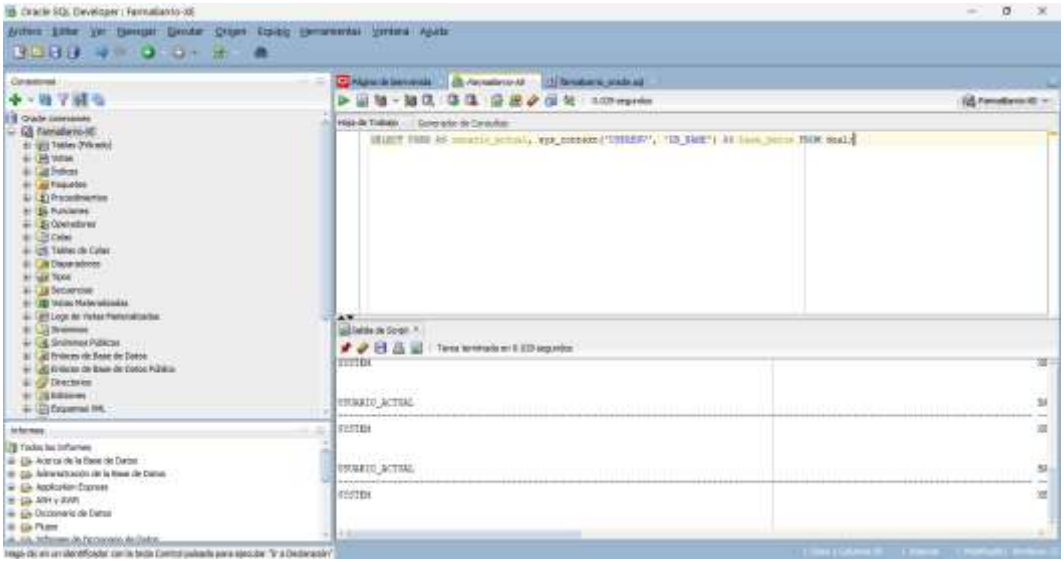
-- Privilegios farma_app (Acceso a tablas operativas de compras y pagos)
GRANT CREATE SESSION TO farma_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON SYSTEM.proveedor TO farma_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON SYSTEM.compra TO farma_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON SYSTEM.detalle_compra TO farma_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON SYSTEM.pagos_proveedor TO farma_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON SYSTEM.documentos_tributarios TO farma_app;

-- Privilegios farma_readonly (Solo lectura para auditoría y reportes)
GRANT CREATE SESSION TO farma_readonly;
GRANT SELECT ON SYSTEM.proveedor TO farma_readonly;
GRANT SELECT ON SYSTEM.compra TO farma_readonly;
GRANT SELECT ON SYSTEM.detalle_compra TO farma_readonly;
GRANT SELECT ON SYSTEM.pagos_proveedor TO farma_readonly;
GRANT SELECT ON SYSTEM.asiento_contable TO farma_readonly;
GRANT SELECT ON SYSTEM.detalle_asiento TO farma_readonly;
```

DESCRIPCION DE PRIVILEGIOS OTORGADOS:

En Oracle Database se estructuró la seguridad mediante roles de sistema y privilegios de objeto granulares: farma_dba posee permisos de administración global de esquemas; farma_app cuenta con privilegios estrictos DML sobre las tablas del módulo transaccional de compras a proveedores con cuota limitada de 500M; y farma_readonly tiene permiso exclusivo de inicio de sesión y consultas SELECT sobre las tablas contables y comerciales para generación de balances sin capacidad de mutación.

CAPTURA DE PANTALLA: (Evidencia de creación de usuarios en Oracle DBA_USERS)



7 RESPALDOS – BACKUPS

7.1 Definir estrategias de respaldos

7.1.1 PostgreSQL

Para el motor PostgreSQL (base de datos farmabarrio_db), se define una estrategia de respaldo mixto compuesta por:

• Frecuencia y Tipo: Respaldo lógico completo semanal (domingos 02:00 a.m.) en formato custom (-F c) mediante pg_dump, complementado con respaldos diarios nocturnos diferenciales. • Política de Retención: Retención mínima de 3 meses en almacenamiento local seguro (C:\backups\postgresql) con sincronización automática a repositorio secundario en la nube. • Ventana de Mantenimiento y Recuperación: Horario de baja transaccionalidad comercial para evitar bloqueos. Se asegura un RTO (Recovery Time Objective) menor a 30 minutos mediante pg_restore y un RPO (Recovery Point Objective) inferior a 24 horas.

7.1.2 Oracle

Para el motor Oracle Database (módulo transaccional de compras a proveedores), se implementa una estrategia basada en Oracle Data Pump:

• Frecuencia y Alcance: Exportación lógica semanal a nivel de esquema completo mediante expdp sobre el directorio DATA_PUMP_DIR, generando archivos binarios .dmp comprimidos y registros detallados .log. • Política de Retención: Mantener activos los últimos 4 respaldos semanales completos (1 mes operativo) y archivar una copia mensual histórica con retención de 1 año. • Verificación y Restauración: Pruebas de restauración periódicas mediante impdp con TABLE_EXISTS_ACTION=REPLACE en entornos de prueba, garantizando integridad referencial y disponibilidad inmediata.

7.1.3 MongoDB

7.1.4 Cassandra

7.2 Scripts de ejecución de backups

BASE DE DATOS:	PostgreSQL

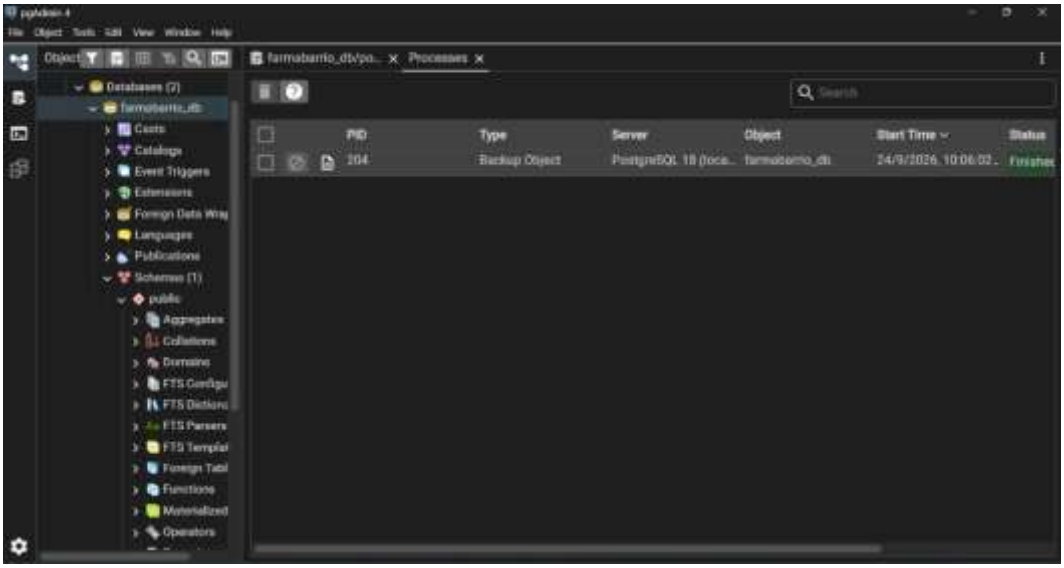
CODIGO DE EJECUCION (RESPALDO Y RESTAURACION):

```
# 1. Ejecución de Respaldo Completo con pg_dump (Formato binario comprimido custom)
pg_dump -U farma_dba -h localhost -p 5432 -d farmabarrio_db -F c -b -v -f
"C:\backups\farmabarrio_full.backup"

# 2. Verificación de archivo generado
dir C:\backups\farmabarrio_full.backup

# 3. Comando de Restauración con pg_restore
pg_restore -U farma_dba -h localhost -p 5432 -d farmabarrio_db -v
"C:\backups\farmabarrio_full.backup"
```

RESULTADO (CAPTURA):



BASE DE DATOS:	Oracle

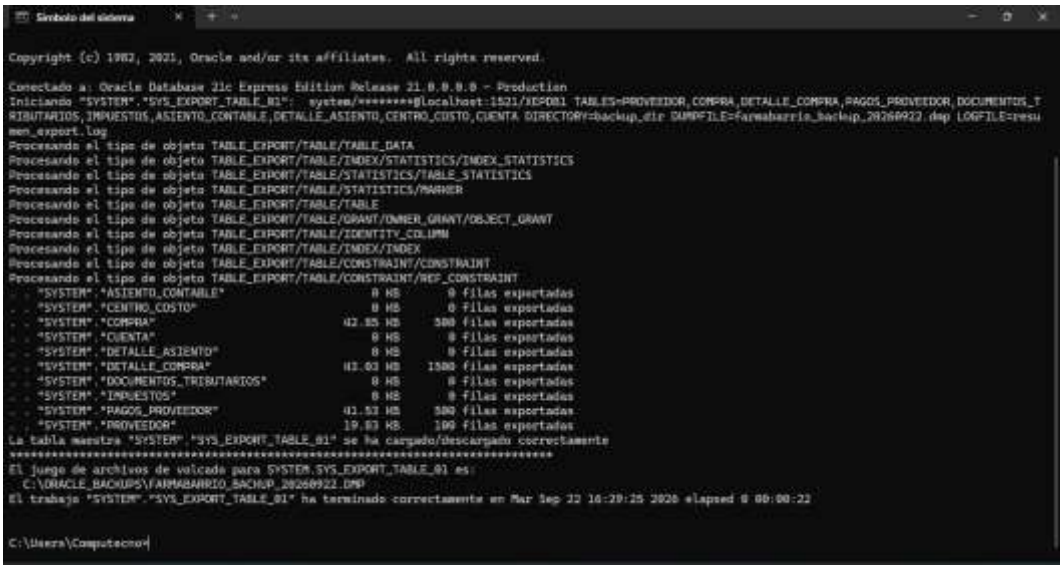
CODIGO DE EJECUCION (ORACLE DATA PUMP):

```
-- 1. Configuración de directorio en Oracle SQL*Plus (como SYSTEM o DBA)
CREATE OR REPLACE DIRECTORY DATA_PUMP_DIR AS 'C:\backups\oracle';
GRANT READ, WRITE ON DIRECTORY DATA_PUMP_DIR TO farma_dba;

# 2. Ejecución de Respaldo Lógico con expdp (Esquema SYSTEM / farmabarrio)
expdp farma_dba/"Dba_2026!"@localhost:1521/XEPDB1 SCHEMAS=SYSTEM DIRECTORY=DATA_PUMP_DIR
DUMPFILE=farmabarrio_backup_%U.dmp LOGFILE=resumen_export.log JOB_NAME=exp_farmabarrio

# 3. Comando de Importación / Restauración con impdp
impdp farma_dba/"Dba_2026!"@localhost:1521/XEPDB1 SCHEMAS=SYSTEM DIRECTORY=DATA_PUMP_DIR
DUMPFILE=farmabarrio_backup_%U.dmp LOGFILE=resumen_import.log TABLE_EXISTS_ACTION=REPLACE
```

RESULTADO (CAPTURA):



BASE DE DATOS:	MongoDB
CODIGO:	
RESULTADO (CAPTURA):	

BASE DE DATOS:	Cassandra
CODIGO:	
RESULTADO (CAPTURA):	

8 BIG DATA

Aplicación de BigData – hadoop

Integración de las 4 base de datos en Haddop

9 Conclusiones y Lecciones aprendidas

9.1 Conclusiones

9.2 Lecciones aprendidas

ANEXOS

Anexo 1: Scripts SQL PostgreSQL (farmabarrio_postgres_ddl_dml.sql)

Archivo de script completo que contiene el código DDL para la creación de la base de datos farmabarrio_db, definición de las 11 tablas con sus llaves primarias, foráneas, restricciones de chequeo, índices compuestos y funciones/procedimientos PL/pgSQL desarrollados, así como el poblado de datos masivo.

Anexo 2: Scripts SQL Oracle (farmabarrio_oracle_ddl_dml.sql)

Archivo de script completo que contiene el código DDL y DML para el gestor Oracle Database, incluyendo la definición de las 10 tablas del módulo contable y comercial, restricciones, secuencias de claves primarias, procedimientos almacenados y funciones PL/SQL para compras y pagos a proveedores.

Anexo 3: Diccionario de Datos Excel (Diccionario_datos_farmabarrio.xlsx)

Documento estructurado en formato Microsoft Excel que consolida el catálogo técnico integral de las 21 tablas del sistema FarmaBarrio (11 tablas en PostgreSQL y 10 tablas en Oracle), describiendo cada campo, tipo de dato nativo, longitud, nulabilidad, valores por defecto, roles de clave (PK, FK, UK) y su funcionalidad de negocio.

Anexo 4: Project Charter (Project_Charter_FarmaBarrio.pdf)

Documento de constitución formal del proyecto FarmaBarrio en el que se detallan el propósito, justificación de negocio, objetivos estratégicos y específicos, alcance, entregables clave, equipo de trabajo con sus roles, cronograma de hitos principales, presupuesto y análisis inicial de riesgos.

Anexo 5: Business Model Canvas (Business_Model_Canvas_FarmaBarrio.pdf)

Lienzo estratégico del modelo de negocio de FarmaBarrio que describe la propuesta de valor centrada en las farmacias independientes y los usuarios finales, segmentos de clientes, canales digitales y logísticos, fuentes de ingresos, estructura de costos y alianzas estratégicas con laboratorios y distribuidores.

Anexo 6: Cronograma de Actividades (Cronograma_Proyecto_FarmaBarrio.pdf)

Planificación detallada de las actividades y entregables del curso Base de Datos II estructurada en un diagrama de Gantt, abarcando las fases de análisis, modelado conceptual y lógico, implementación física en motores relacionales y NoSQL, fragmentación, replicación y preparación de sustentación.